

Fəsil 11. Dərin Neyron Şəbəkələrin Təlimi

Fəsil 10-da siz ilk süni neyron şəbəkələrinizi qurdunuz, öyrətdiniz və incə tənzimlədiniz. Lakin bunlar dayaz şəbəkələr idi, yalnız bir neçə gizli qatla. Bəs yüksək dəqiqlikli şəkillərdə yüzlərlə növ obyektı aşkar etmək kimi mürəkkəb bir problemi həll etmək lazımdırsa? Sizə daha dərin bir ANN, bəlkə 10 qat və ya daha çox, hər birində yüzlərlə neyron və yüz minlərlə əlaqə olan bir şəbəkə lazım ola bilər. Dərin neyron şəbəkəni (DNN) öyrətmək asan iş deyil. Qarşılaşa biləcəyiniz bəzi problemlər bunlardır:

- Təlim zamanı DNN-dən geriye doğru axan qradiyentlər getdikcə kiçilə və ya böyüyə bilər. Bu problemlərin hər ikisi aşağı qatların öyrənilməsini çox çətinləşdirir.
- Belə böyük bir şəbəkə üçün kifayət qədər təlim məlumatınız olmaya bilər və ya onları etiketləmək çox baha başa gələ bilər.
- Təlim son dərəcə yavaş ola bilər.
- Milyonlarla parametreyə malik bir model, xüsusən də kifayət qədər təlim nümunəsi yoxdursa və ya onlar çox səssizdirsə, təlim setini aşırı uyğunlaşdırma (overfitting) riskini ciddi şəkildə artırır.

Bu fəsildə biz bu problemlərin hər birini nəzərdən keçirəcək və onları həll etmək üçün texnikalar təqdim edəcəyik. Sönən və partlayan qradiyent problemləri və onların ən populyar həll yollarını araşdırmaqla başlayacağıq. Sonra, az miqdarda etiketli məlumatınız olsa belə mürəkkəb tapşırıqları həll etməyə kömək edə bilən transfer learning və unsupervised pretraining-ə baxacağıq. Daha sonra böyük modellərin təlimini nəhəng dərəcədə sürətləndirə bilən müxtəlif optimizatorları müzakirə edəcəyik. Nəhayət, böyük neyron şəbəkələr üçün bir neçə məşhur regularizasiya texnikasını əhatə edəcəyik.

Bu alətlərlə siz çox dərin şəbəkələri öyrədə biləcəksiniz. Deep Learning-ə xoş gəlmisiniz!

Sönən/Partlayan Qradiyent Problemləri

Fəsil 10-da müzakirə edildiyi kimi, backpropagation alqoritminin ikinci mərhələsi çıxış qatından giriş qatına doğru işləyir və yol boyu səhv qradiyentini yayır. Alqoritm şəbəkədəki hər bir parametreyə görə məsrəf funksiyasının qradiyentini hesabladıqdan sonra, bu qradiyentlərdən istifadə edərək hər bir parametri gradient descent addımı ilə yeniləyir.

Təəssüf ki, qradiyentlər alqoritm aşağı qatlara doğru irəlilədikcə tez-tez kiçilir. Nəticədə, gradient descent yeniləməsi aşağı qatların birləşmə çəkirlərini demək olar

ki, dəyişməz qoyur və təlim heç vaxt yaxşı bir həllə yığılmır. Buna *vanishing gradients* (sönən qradiyentlər) problemi deyilir. Bəzi hallarda əks vəziyyət baş verə bilər: qradiyentlər böyüyüb böyüyə bilər, nəticədə qatlar çox böyük çəki yeniləmələri alır və alqoritm ayrılır (diverges). Bu, *exploding gradients* (partlayan qradiyentlər) problemidir və ən çox recurrent neural networks (bax Fəsil 15) zamanı ortaya çıxır. Ümumiyyətlə, dərin neyron şəbəkələr qeyri-sabit qradiyentlərdən əziyyət çəkir; müxtəlif qatlar çox fərqli sürətlərdə öyrənə bilər.

Bu arzuolunmaz davranış çoxdan empirik olaraq müşahidə edilmişdir və 2000-ci illərin əvvəllərində dərin neyron şəbəkələrin əsasən tərk edilməsinin səbəblərindən biri olmuşdur. DNN təlimi zamanı qradiyentlərin niyə bu qədər qeyri-sabit olduğu aydın deyildi, lakin 2010-cu ildə Xavier Glorot və Yoshua Bengio-nun məqaləsində bəzi işıqlar göründü. Müəlliflər bir neçə şübhəli tapdılar, o cümlədən məşhur sigmoid (logistik) aktivasiya funksiyasının o vaxt ən populyar olan çəki ilkinləşdirmə texnikası ilə (yəni, ortalama 0 və standart sapma 1 olan normal paylanma) birləşməsi. Qısacası, onlar göstərdilər ki, bu aktivasiya funksiyası və ilkinləşdirmə sxemi ilə hər bir qatın çıxışlarının dispersiyası, onun girişlərinin dispersiyasından çox daha böyükdür. Şəbəkədə irəli getdikcə, dispersiya hər qatdan sonra artır, nəhayət aktivasiya funksiyası yuxarı qatlarda doymur (saturates). Bu doyma, sigmoid funksiyasının ortalamasının 0.5 olması (0 deyil) ilə daha da pisləşir (hiperbolik tangens funksiyasının ortalaması 0-dır və dərin şəbəkələrdə sigmoid funksiyasından bir qədər yaxşı davranır).

Sigmoid aktivasiya funksiyasına baxdıqda (bax Şəkil 11-1), girişlər böyük (mənfi və ya müsbət) olduqda funksiyanın 0 və ya 1-də doyduğunu və törəməsinin son dərəcə 0-a yaxın olduğunu görə bilərsiniz (yəni əyri hər iki ucda düzdür). Beləliklə, backpropagation işə düşdükdə, şəbəkə vasitəsilə geri yayılacaq demək olar ki, heç bir qradiyent yoxdur və mövcud olan azacıq qradiyent də yuxarı qatlardan aşağıya doğru irəlilədikcə seyrəldilir, buna görə də aşağı qatlar üçün heç nə qalmır.

Şəkil 11-1. Sigmoid aktivasiya funksiyasının doyması

Glorot və He ilkinləşdirməsi

Məqalələrində Glorot və Bengio qeyri-sabit qradiyentlər problemini əhəmiyyətli dərəcədə azaltmağın bir yolunu təklif edirlər. Onlar qeyd edirlər ki, signal hər iki istiqamətdə düzgün axmalıdır: proqnozlar verərkən irəli istiqamətdə və qradiyentləri geri yayarkən əks istiqamətdə. Signalın nə ölməsini, nə də partlayıb doymasını istəmirik. Signalın düzgün axması üçün müəlliflər iddia edirlər ki, hər bir qatın çıxışlarının dispersiyası onun girişlərinin dispersiyasına bərabər olmalıdır və qradiyentlər əks istiqamətdə qatdan keçməzdən əvvəl və sonra bərabər dispersiyaya malik olmalıdır (riyazi detallarla maraqlanırsınızsa, məqaləyə baxın). Qatın bərabər

sayda giriş və çıxışı olmadıqca hər ikisini təmin etmək mümkün deyil (bu ədədlər qatın *fan-in* və *fan-out* adlanır), lakin Glorot və Bengio praktikada çox yaxşı işlədiyi sübut edilmiş yaxşı bir kompromis təklif etdilər: hər bir qatın birləşmə çəkili Tənlik 11-1-də təsvir edildiyi kimi təsadüfi ilkinləşdirilməlidir, burada $\text{fan_avg} = (\text{fan_in} + \text{fan_out}) / 2$. Bu ilkinləşdirmə strategiyası məqalənin ilk müəllifinin şərəfinə *Xavier initialization* və ya *Glorot initialization* adlanır.

Tənlik 11-1. Glorot ilkinləşdirməsi (sigmoid aktivasiya funksiyası istifadə edildikdə)

Ortalama 0 və dispersiya $\sigma^2 = 1 / \text{fan_avg}$ olan normal paylanma və ya $-r$ ilə $+r$ arasında vahid paylanma, burada $r = \sqrt{3 / \text{fan_avg}}$

Əgər Tənlik 11-1-də fan_avg -i fan_in ilə əvəz etsəniz, 1990-cı illərdə Yann LeCun tərəfindən təklif edilən ilkinləşdirmə strategiyasını alırsınız. O, bunu *LeCun initialization* adlandırdı. Genevieve Orr və Klaus-Robert Müller hətta 1998-ci ildəki *Neural Networks: Tricks of the Trade* (Springer) kitablarında bunu tövsiyə etmişdilər. LeCun initialization, $\text{fan_in} = \text{fan_out}$ olduqda Glorot initialization-ə ekvivalentdir. Tədqiqatçıların bu hiylənin nə qədər vacib olduğunu anlaması on ildən çox vaxt apardı. Glorot initialization istifadəsi təlimi əhəmiyyətli dərəcədə sürətləndirə bilər və deep learning-in uğuruna gətirib çıxaran təcrübələrdən biridir.

Bəzi məqalələr müxtəlif aktivasiya funksiyaları üçün oxşar strategiyalar təqdim etmişdir. Bu strategiyalar yalnız dispersiyanın miqyasına və fan_avg və ya fan_in istifadə etmələrinə görə fərqlənir, Cədvəl 11-1-də göstərildiyi kimi (vahid paylanma üçün sadəcə $r = \sqrt{3\sigma^2}$ istifadə edin). ReLU aktivasiya funksiyası və onun variantları üçün təklif edilən ilkinləşdirmə strategiyası məqalənin ilk müəllifinin şərəfinə *He initialization* və ya *Kaiming initialization* adlanır. SELU üçün Yann LeCun-un ilkinləşdirmə metodundan, tercihen normal paylanma ilə istifadə edin. Bütün bu aktivasiya funksiyalarını qısa müddətdə əhatə edəcəyik.

Cədvəl 11-1. Hər bir aktivasiya funksiyası növü üçün ilkinləşdirmə parametrləri

İlkinləşdirmə	Aktivasiya funksiyaları	σ^2 (Normal)	σ^2 (Normal)
Glorot	None, tanh, sigmoid, softmax	$1 / \text{fan_avg}$	
He	ReLU, Leaky ReLU, ELU, GELU, Swish, Mish	$2 / \text{fan_in}$	

Standart olaraq Keras vahid paylanma ilə Glorot ilkinləşdirməsindən istifadə edir. Bir qat yaratdığınız zaman, `kernel_initializer="he_uniform"` və ya `kernel_initializer="he_normal"` təyin etməklə He ilkinləşdirməsinə keçə bilərsiniz:

```
import tensorflow as tf

dense = tf.keras.layers.Dense(50, activation="relu",
kernel_initializer="he_normal")
```

Alternativ olaraq, Cədvəl 11-1-də sadalanan və daha çox ilkinləşdirmələri `VarianceScaling` ilkinləşdiricisindən istifadə edərək əldə edə bilərsiniz. Məsələn, `fan_avg` əsasında vahid paylanma ilə He ilkinləşdirməsini istəyirsinizsə (`fan_in` əvəzinə), aşağıdakı koddan istifadə edə bilərsiniz:

```
he_avg_init = tf.keras.initializers.VarianceScaling(scale=2.,
mode="fan_avg", distribution="uniform")

dense = tf.keras.layers.Dense(50, activation="sigmoid",
kernel_initializer=he_avg_init)
```

Daha Yaxşı Aktivasiya Funksiyaları

Glorot və Bengio-nun 2010-cu il məqaləsindəki fikirlərdən biri, qeyri-sabit qradientlər probleminin qismən pis aktivasiya funksiyası seçimindən qaynaqlandığı idi. O vaxta qədər əksər insanlar Təbiət Ana bioloji neyronlarda təxminən sigmoid aktivasiya funksiyalarından istifadə etməyi seçmişsə, bunun mükəmməl bir seçim olduğunu fərz edirdilər. Lakin məlum oldu ki, digər aktivasiya funksiyaları dərin neyron şəbəkələrdə daha yaxşı davranır – xüsusilə ReLU aktivasiya funksiyası, əsasən müsbət qiymətlər üçün doymadığına və çox sürətli hesablandığına görə.

Təəssüf ki, ReLU aktivasiya funksiyası mükəmməl deyil. O, *dying ReLUs* (ölən ReLU) kimi tanınan bir problemdən əziyyət çəkir: təlim zamanı bəzi neyronlar effektiv olaraq "ölür", yəni 0-dan başqa heç nə çıxarmağı dayandırır. Bəzi hallarda şəbəkənin neyronlarının yarısının ölü olduğunu görə bilərsiniz, xüsusən böyük öyrənmə sürəti (learning rate) istifadə etsəniz. Neyron, onun çəkili elə tənzimləndikdə ölür ki, ReLU funksiyasının girişi (yəni neyronun girişlərinin çəkili cəmi üstəgəl bias termini) təlim setindəki bütün instanslar üçün mənfi olur. Bu baş verdikdə, o, yalnız sıfırlar

çıxarmağa davam edir və gradient descent artıq ona təsir etmir, çünki ReLU funksiyasının qradienti girişi mənfi olduqda sıfırdır.

Bu problemi həll etmək üçün ReLU funksiyasının bir variantından, məsələn *leaky ReLU* -dan istifadə etmək istəyə bilərsiniz.

Leaky ReLU

Leaky ReLU aktivasiya funksiyası $\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z)$ kimi təyin olunur (bax Şəkil 11-2). Hiperparametr α funksiyanın nə qədər "sızdığını" müəyyən edir: bu, $z < 0$ üçün funksiyanın yamacıdır. $z < 0$ üçün bir yamaca sahib olmaq, leaky ReLU-ların heç vaxt ölməməsini təmin edir; onlar uzun bir komaya girə bilər, lakin nəhayət oyanmaq şansları var. Bing Xu və digərlərinin 2015-ci il məqaləsi ReLU aktivasiya funksiyasının bir neçə variantını müqayisə etdi və nəticələrindən biri, leaky variantların həmişə sərt ReLU aktivasiya funksiyasından daha yaxşı çıxış etməsi idi. Əslində, $\alpha = 0.2$ (böyük sızma) təyin etmək $\alpha = 0.01$ (kiçik sızma) ilə müqayisədə daha yaxşı performans səbəb oldu. Məqalə həmçinin *randomized leaky ReLU* (RReLU) qiymətləndirdi, burada α təlim zamanı verilmiş diapazonda təsadüfi seçilir və test zamanı ortalama dəyərə sabitlənir. RReLU da kifayət qədər yaxşı çıxış etdi və regularizator kimi çıxış edərək overfitting riskini azaltdı. Nəhayət, məqalə *parametric leaky ReLU* (PReLU) qiymətləndirdi, burada α -nın təlim zamanı öyrənilməsinə icazə verilir: hiperparametr əvəzinə, o, backpropagation tərəfindən hər hansı digər parametr kimi dəyişdirilə bilən bir parametr olur. PReLU-nun böyük şəkil dataset-lərində ReLU-dan güclü şəkildə üstün olduğu bildirildi, lakin kiçik dataset-lərdə overfitting riski daşıyır.

Şəkil 11-2. *Leaky ReLU*: ReLU kimi, lakin mənfi dəyərlər üçün kiçik bir yamaca

Keras `tf.keras.layers` paketində `LeakyReLU` və `PReLU` siniflərini ehtiva edir. Digər ReLU variantları kimi, bunlarla He ilkinləşdirməsindən istifadə etməlisiniz. Məsələn:

```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2) # default alpha=0.3

dense = tf.keras.layers.Dense(50, activation=leaky_relu,
kernel_initializer="he_normal")
```

İstəyirsinizsə, `LeakyReLU`-nu modeldə ayrı bir qat kimi də istifadə edə bilərsiniz; təlim və proqnozlar üçün heç bir fərq yoxdur:

```
model = tf.keras.models.Sequential([
    [...] # more layers
    tf.keras.layers.Dense(50, kernel_initializer="he_normal"), # no
activation
```

```
tf.keras.layers.LeakyReLU(alpha=0.2), # activation as a separate layer
[... ] # more layers

])
```

PReLU üçün LeakyReLU-nu PReLU ilə əvəz edin. Keras-da RReLU-nun rəsmi tətbiqi hələ yoxdur, lakin siz özünüz asanlıqla tətbiq edə bilərsiniz (bunu necə edəcəyinizi öyrənmək üçün Fəsil 12-nin sonundakı məşqlərə baxın).

ReLU, leaky ReLU və PReLU-nun hamısı hamar (smooth) funksiyalar olmamaqdan əziyyət çəkir: onların törəmələri ($z = 0$ -da) qəfil dəyişir. Fəsil 4-də lasso-nu müzakirə edərkən gördüyümüz kimi, bu cür kəsilmə gradient descent-in optimal ətrafında sıçramasına və yığılmanı yavaşlatmasına səbəb ola bilər. İndi ReLU aktivasiya funksiyasının bəzi hamar variantlarına baxaq, ELU və SELU ilə başlayaraq.

ELU və SELU

2015-ci ildə Djork-Arné Clevert və digərlərinin məqaləsi *exponential linear unit* (ELU) adlı yeni aktivasiya funksiyası təklif etdi və müəlliflərin təcrübələrində bütün ReLU variantlarından üstün oldu: təlim müddəti azaldı və neyron şəbəkə test setində daha yaxşı performans göstərdi. Tənlik 11-2 bu aktivasiya funksiyasının tərifini göstərir.

Tənlik 11-2. ELU aktivasiya funksiyası

$$\text{ELU}_\alpha(z) = \alpha(\exp(z) - 1) \text{ əgər } z < 0, z \text{ əgər } z \geq 0$$

ELU aktivasiya funksiyası ReLU funksiyasına çox bənzəyir (bax Şəkil 11-3), lakin bir neçə əsas fərqlə:

- $z < 0$ olduqda mənfi qiymətlər alır, bu da vahidin ortalama çıxışının 0-a yaxınlaşmasına imkan verir və vanishing gradients problemini yüngülləşdirməyə kömək edir.
- α hiperparametri ELU funksiyasının z böyük mənfi ədəd olduqda yaxınlaşdığı dəyərin əksini müəyyən edir. Adətən 1-ə təyin edilir, lakin digər hiperparametrlər kimi onu da tənzimləyə bilərsiniz.
- $z < 0$ üçün sıfırdan fərqli qradientə malikdir, bu da ölü neyronlar probleminin qarşısını alır.
- $\alpha = 1$ olduqda funksiya hər yerdə, o cümlədən $z = 0$ ətrafında hamardır, bu da gradient descent-i sürətləndirir, çünki $z = 0$ -ın soluna və sağına çox sıçramır.

ELU-nu Keras-da istifadə etmək `activation="elu"` təyin etmək qədər sadədir və digər ReLU variantları kimi He ilkinləşdirməsindən istifadə etməlisiniz. ELU aktivasiya funksiyasının əsas çatışmazlığı onun ReLU funksiyası və variantlarından daha yavaş

hesablanmasıdır (eksponensial funksiyanın istifadəsi səbəbindən). Təlim zamanı daha sürətli yığılma sürəti bu yavaş hesablamanı kompensasiya edə bilər, lakin test zamanı ELU şəbəkəsi ReLU şəbəkəsindən bir qədər yavaş olacaq.

Şəkil 11-3. ELU və SELU aktivasiya funksiyaları

Çox keçmədən, 2017-ci ildə Günter Klambauer və digərlərinin məqaləsi *scaled ELU* (SELU) aktivasiya funksiyasını təqdim etdi: adından göründüyü kimi, bu, ELU aktivasiya funksiyasının miqyaslanmış variantıdır (təxminən 1.05 dəfə ELU, $\alpha \approx 1.67$ istifadə edərək). Müəlliflər göstərdilər ki, əgər siz yalnız sıx qatlar yığınınından (yəni MLP) ibarət bir neyron şəbəkəsi qursanız və bütün gizli qatlar SELU aktivasiya funksiyasından istifadə edərsə, şəbəkə *self-normalize* (öz-özünü normallaşdıracaq): hər bir qatın çıxışı təlim zamanı ortalama 0 və standart sapma 1-i qorumağa meylli olacaq, bu da vanishing/exploding gradients problemini həll edir. Nəticədə, SELU aktivasiya funksiyası MLP-lər, xüsusən də dərin olanlar üçün digər aktivasiya funksiyalarından üstün ola bilər. Keras-da istifadə etmək üçün sadəcə `activation="selu"` təyin edin. Bununla belə, self-normalizasiyanın baş verməsi üçün bir neçə şərt var (riyazi əsaslandırma üçün məqaləyə baxın):

- Giriş xüsusiyyətləri standartlaşdırılmış olmalıdır: ortalama 0 və standart sapma 1.
- Hər bir gizli qatın çəkilişi LeCun normal ilkinləşdirməsindən istifadə edərək ilkinləşdirilməlidir. Keras-da bu, `kernel_initializer="lecun_normal"` təyin etmək deməkdir.
- Self-normalizasiya xüsusiyyəti yalnız sadə MLP-lərlə təmin edilir. Əgər SELU-nu digər arxitekturalarda, məsələn, recurrent networks (bax Fəsil 15) və ya *skip connections* (qatları atlayan əlaqələr, məsələn, Wide & Deep nets) olan şəbəkələrdə istifadə etməyə çalışsanız, yəqin ki, ELU-dan üstün olmayacaq.
- ℓ_1 və ya ℓ_2 regularizasiya, max-norm, batch-norm və ya adi dropout kimi regularizasiya texnikalarından istifadə edə bilməzsiniz (bunlar bu fəsildə sonra müzakirə olunur).

Bunlar əhəmiyyətli məhdudiyyətlərdir, buna görə də vədlərinə baxmayaraq, SELU geniş yayılmadı. Üstəlik, daha üç aktivasiya funksiyası əksər tapşırıqlarda ondan kifayət qədər ardıcıl olaraq üstün görünür: GELU, Swish və Mish.

GELU, Swish və Mish

GELU 2016-cı ildə Dan Hendrycks və Kevin Gimpel tərəfindən təqdim edilmişdir. Yenə də onu ReLU aktivasiya funksiyasının hamar bir variantı kimi düşünə bilərsiniz. Onun tərifini Tənlik 11-3-də verilmişdir, burada Φ standart Gaussian kumulyativ paylanma

funksiyasıdır (CDF): $\Phi(z)$ ortalama 0 və dispersiya 1 olan normal paylanmadan təsadüfi seçilmiş dəyərin z -dən kiçik olma ehtimalına uyğundur.

Tənlilik 11-3. GELU aktivasiya funksiyası

$$\text{GELU}(z) = z \Phi(z)$$

Şəkil 11-4-də gördüyünüz kimi, GELU ReLU-ya bənzəyir: girişi z çox mənfi olduqda 0-a yaxınlaşır və z çox müsbət olduqda z -yə yaxınlaşır. Bununla belə, indiyə qədər müzakirə etdiyimiz bütün aktivasiya funksiyaları həm qabarıq (convex), həm də monotonik olduğu halda, GELU aktivasiya funksiyası nə qabarıq, nə də monotonikdir: soldan sağa, əvvəl düz gedir, sonra aşağı əyilir, təxminən -0.17 ətrafında ($z \approx -0.75$ yaxınlığında) ən aşağı nöqtəyə çatır və nəhayət yuxarı sıçrayıb sağ yuxarıya doğru düz gedir. Bu kifayət qədər mürəkkəb forma və onun hər nöqtədə əyriyə malik olması, xüsusən mürəkkəb tapşırıqlar üçün niyə bu qədər yaxşı işlədiyini izah edə bilər: gradient descent mürəkkəb naxışları uyğunlaşdırmağı daha asan tapa bilər. Praktikada tez-tez indiyə qədər müzakirə olunan hər bir digər aktivasiya funksiyasından üstündür. Lakin o, bir qədər daha çox hesablama tələb edir və təmin etdiyi performans artımı həmişə əlavə xərci əsaslandırmaq üçün kifayət etmir. Bununla belə, onun təxminən $z \sigma(1.702 z)$ -yə bərabər olduğunu göstərmək mümkündür, burada σ sigmoid funksiyasıdır: bu yaxınlaşmadan istifadə də çox yaxşı işləyir və daha sürətli hesablama üstünlüyünə malikdir.

Şəkil 11-4. GELU, Swish, parametrizə edilmiş Swish və Mish aktivasiya funksiyaları

GELU məqaləsi həmçinin *sigmoid linear unit* (SiLU) aktivasiya funksiyasını təqdim etdi, bu $z \sigma(z)$ -yə bərabərdir, lakin müəlliflərin testlərində GELU-dan üstün deyildi. Maraqlıdır ki, 2017-ci ildə Prajit Ramachandran və digərlərinin məqaləsi yaxşı aktivasiya funksiyalarını avtomatik axtararaq SiLU funksiyasını yenidən kəşf etdi. Müəlliflər onu *Swish* adlandırdılar və bu ad qəbul olundu. Məqalələrində Swish GELU da daxil olmaqla bütün digər funksiyalardan üstün oldu. Ramachandran və digərləri daha sonra sigmoid funksiyasının girişini miqyaslandırmaq üçün əlavə bir β hiperparametri əlavə edərək Swish-i ümumiləşdirdilər. Ümumiləşdirilmiş Swish funksiyası $\text{Swish}_\beta(z) = z \sigma(\beta z)$ kimi verilir, buna görə GELU təxminən $\beta = 1.702$ istifadə edən ümumiləşdirilmiş Swish funksiyasına bərabərdir. β -ni hər hansı digər hiperparametr kimi tənzimləyə bilərsiniz. Alternativ olaraq, β -ni öyrənilə bilən etmək və gradient descent-ə onu optimallaşdırmağa icazə vermək mümkündür: PReLU kimi, bu modelinizi daha güclü edə bilər, lakin eyni zamanda məlumatları aşırı uyğunlaşdırma riskini daşıyır.

Digər olduqca oxşar aktivasiya funksiyası *Mish* adlanır və 2019-cu ildə Diganta Misra tərəfindən təqdim edilmişdir. O, $\text{mish}(z) = z \tanh(\text{softplus}(z))$ kimi təyin olunur, burada $\text{softplus}(z) = \log(1 + \exp(z))$. GELU və Swish kimi, bu da ReLU-nun hamar, qabarıq

olmayan və monotonik olmayan variantıdır və yenə müəllif çoxsaylı təcrübələr apardı və Mish-in ümumiyyətlə digər aktivasiya funksiyalarından – hətta Swish və GELU-dan da az fərqlə üstün olduğunu tapdı. Şəkil 11-4 GELU, Swish (həm standart $\beta = 1$, həm də $\beta = 0.6$ ilə) və nəhayət Mish-i göstərir. Gördüyünüz kimi, Mish z mənfi olduqda Swish ilə demək olar ki, mükəmməl üst-üstə düşür və z müsbət olduqda GELU ilə demək olar ki, mükəmməl üst-üstə düşür.

İPUCU

Beləliklə, dərin neyron şəbəkələrinizin gizli qatları üçün hansı aktivasiya funksiyasından istifadə etməlisiniz? ReLU sadə tapşırıqlar üçün yaxşı bir seçim olaraq qalır: çox vaxt daha mürəkkəb aktivasiya funksiyaları qədər yaxşıdır, üstəlik çox sürətli hesablanır və bir çox kitabxanalar və aparat sürətləndiriciləri ReLU-ya xüsusi optimizasiyalar təqdim edir. Lakin Swish daha mürəkkəb tapşırıqlar üçün daha yaxşı bir defolt ola bilər və hətta ən mürəkkəb tapşırıqlar üçün öyrənilə bilən β parametri ilə parametrizə edilmiş Swish-i sınağa bilərsiniz. Mish bir qədər yaxşı nəticələr verə bilər, lakin bir qədər daha çox hesablama tələb edir. Əgər icra gecikməsi (runtime latency) sizin üçün çox vacibdirsə, leaky ReLU-ya və ya daha mürəkkəb tapşırıqlar üçün parametrizə edilmiş leaky ReLU-ya üstünlük verə bilərsiniz. Dərin MLP-lər üçün SELU-nu sınayın, lakin əvvəllər sadalanan məhdudiyyətlərə riayət etdiyinizə əmin olun. Əgər boş vaxtınız və hesablama gücünüz varsa, cross-validation istifadə edərək digər aktivasiya funksiyalarını da qiymətləndirə bilərsiniz.

Keras GELU və Swish-i hazır dəstəkləyir; sadəcə `activation="gelu"` və ya `activation="swish"` istifadə edin. Bununla belə, o, Mish və ya ümumiləşdirilmiş Swish aktivasiya funksiyasını hələ dəstəkləmir (lakin öz aktivasiya funksiyalarınızı və qatlarınızı necə tətbiq edəcəyinizi görmək üçün Fəsil 12-yə baxın).

Aktivasiya funksiyaları ilə bağlı hamısı budur! İndi qeyri-sabit qradiyentlər problemini həll etmək üçün tamamilə fərqli bir üsula baxaq: batch normalization.

Batch Normalization

He ilkinləşdirməsini ReLU (və ya onun hər hansı variantı) ilə birlikdə istifadə etmək təlimin əvvəlində vanishing/exploding gradients təhlükəsini əhəmiyyətli dərəcədə azalda bilsə də, onların təlim zamanı geri qayıtmayacağına zəmanət vermir.

2015-ci ildə Sergey Ioffe və Christian Szegedy tərəfindən təklif edilən *batch normalization* (BN) texnikası bu problemləri həll edir. Texnika hər bir gizli qatın aktivasiya funksiyasından əvvəl və ya sonra modelə bir əməliyyat əlavə etməkdən ibarətdir. Bu əməliyyat sadəcə hər bir girişi sıfır-mərkəzləşdirir və normallaşdırır,

sonra nəticəni hər bir qat üçün iki yeni parametr vektorundan istifadə edərək miqyaslandırır və sürüşdürür: biri miqyaslandırma üçün, digəri sürüşdürmə üçün. Başqa sözlə, bu əməliyyat modelə hər bir qatın girişlərinin optimal miqyasını və ortalamasını öyrənməyə imkan verir. Əgər BN qatını neyron şəbəkənin ilk qatı kimi əlavə etsəniz, təlim setinizi standartlaşdırmağa ehtiyac qalmır. Yəni, StandardScaler və ya Normalization-a ehtiyac yoxdur; BN qatı bunu sizin üçün edəcək (təqribən, çünki o, yalnız bir batch-ə baxır və hər bir giriş xüsusiyyətini yenidən miqyaslandırır və sürüşdürə bilər).

Girişləri sıfır-mərkəzləşdirmək və normallaşdırmaq üçün alqoritm hər bir girişin ortalaması və standart sapmasını qiymətləndirməlidir. O, bunu cari mini-batch üzərindən girişin ortalaması və standart sapmasını hesablamaqla edir (buna görə "batch normalization" adı). Bütün əməliyyat Tənlik 11-4-də addım-addım ümumiləşdirilmişdir.

Tənlik 11-4. Batch normalization alqoritmi

1. $\mu_B = (1/m_B) \sum_{i=1}^{m_B} x^{(i)}$
2. $\sigma_B^2 = (1/m_B) \sum_{i=1}^{m_B} (x^{(i)} - \mu_B)^2$
3. $\tilde{x}^{(i)} = (x^{(i)} - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$
4. $z^{(i)} = \gamma \otimes \tilde{x}^{(i)} + \beta$

Bu alqoritmə:

- μ_B giriş ortalamalarının vektorudur, bütün mini-batch B üzərində qiymətləndirilir (hər giriş üçün bir ortalamaya ehtiva edir).
- m_B mini-batch-dəki instansların sayıdır.
- σ_B giriş standart sapmalarının vektorudur, yenə bütün mini-batch üzərində qiymətləndirilir (hər giriş üçün bir standart sapmaya ehtiva edir).
- $\tilde{x}^{(i)}$ sıfır-mərkəzləşdirilmiş və normallaşdırılmış giriş vektorudur.
- ϵ sıfıra bölmənin qarşısını alan və qradientlərin çox böyüməməsini təmin edən kiçik bir ədəddir (adətən 10^{-5}). Buna *smoothing term* deyilir.
- γ qat üçün çıxış miqyas parametrləri vektorudur (hər giriş üçün bir miqyas parametri ehtiva edir).
- $\otimes$ element-wise vurmanı təmsil edir (hər bir giriş müvafiq çıxış miqyas parametri ilə vurulur).
- β qat üçün çıxış sürüşdürmə (offset) parametrləri vektorudur (hər giriş üçün bir offset parametri ehtiva edir). Hər bir giriş müvafiq sürüşdürmə parametri ilə offset edilir.
- $z^{(i)}$ BN əməliyyatının çıxışıdır. Bu, girişlərin yenidən miqyaslandırılmış və offset edilmiş versiyasıdır.

Beləliklə, təlim zamanı BN öz girişlərini standartlaşdırır, sonra yenidən miqyaslandırır və offset edir. Test zamanı isə nə olur? Bu o qədər də sadə deyil. Həqiqətən də, batch-lər əvəzinə fərdi instanslar üçün proqnozlar vermək lazım ola bilər: bu halda, hər bir girişin ortalama və standart sapmasını hesablamaq üçün heç bir yol olmayacaq. Üstəlik, batch instanslarımız olsa belə, çox kiçik ola bilər və ya instanslar müstəqil və eyni paylanmış olmaya bilər, buna görə batch instansları üzərində statistikalar hesablamaq etibarsız olardı. Bir həll təlimin sonuna qədər gözləmək, sonra bütün təlim setini neyron şəbəkədən keçirmək və BN qatının hər bir girişinin ortalama və standart sapmasını hesablamaq ola bilər. Bu "son" giriş ortalamaları və standart sapmaları daha sonra proqnozlar verərkən batch giriş ortalamaları və standart sapmaları əvəzinə istifadə edilə bilər. Bununla belə, batch normalization-un əksər tətbiqləri bu son statistikaları təlim zamanı qatın giriş ortalamalarının və standart sapmalarının hərəkətli ortalamasını istifadə edərək qiymətləndirir. Keras `BatchNormalization` qatından istifadə etdikdə bunu avtomatik edir. Xülasə, hər bir batch-normalized qatda dörd parametr vektoru öyrənilir: γ (çıxış miqyas vektoru) və β (çıxış offset vektoru) adi backpropagation vasitəsilə öyrənilir, μ (son giriş ortalama vektoru) və σ (son giriş standart sapma vektoru) isə eksponensial hərəkətli ortalama istifadə edərək qiymətləndirilir. Qeyd edək ki, μ və σ təlim zamanı qiymətləndirilir, lakin yalnız təlimdən sonra istifadə olunur (Tənlik 11-4-dəki batch giriş ortalamaları və standart sapmalarını əvəz etmək üçün).

Ioffe və Szegedy batch normalization-un təcrübə apardıqları bütün dərin neyron şəbəkələri əhəmiyyətli dərəcədə yaxşılaşdırdığını, ImageNet təsnifat tapşırığında böyük təkmilləşməyə səbəb olduğunu nümayiş etdirdilər (ImageNet çoxlu sayda sinfə təsnif edilmiş böyük şəkillər bazasıdır, kompüter görmə sistemlərini qiymətləndirmək üçün geniş istifadə olunur). Vanishing gradients problemi o qədər azaldı ki, onlar doymuş aktivasiya funksiyalarından, məsələn tanh və hətta sigmoid funksiyasından istifadə edə bildilər. Şəbəkələr həmçinin çəki ilkinləşdirməsinə daha az həssas idilər. Müəlliflər daha böyük öyrənmə sürətlərindən istifadə edə bildilər ki, bu da öyrənmə prosesini əhəmiyyətli dərəcədə sürətləndirdi. Xüsusilə, onlar qeyd edirlər:

"Ən müasir şəkil təsnifat modelinə tətbiq edildikdə, batch normalization eyni dəqiqliyi 14 dəfə az təlim addımı ilə əldə edir və orijinal modeli əhəmiyyətli dərəcədə üstələyir. [...] Batch-normalized şəbəkələr ansamblından istifadə edərək, ImageNet təsnifatında ən yaxşı nəşr edilmiş nəticəni yaxşılaşdırırıq: 4.9% top-5 validasiya səhvi (və 4.8% test səhvi) əldə edərək, insan qiymətləndiricilərinin dəqiqliyini aşırıq."

Nəhayət, davamlı hədiyyə kimi, batch normalization eyni zamanda regularizator kimi çıxış edir, digər regularizasiya texnikalarına (məsələn, bu fəsildə sonra təsvir olunan dropout) ehtiyacı azaldır.

Bununla belə, batch normalization modelə müəyyən mürəkkəblilik əlavə edir (baxmayaraq ki, əvvəllər müzakirə edildiyi kimi, giriş məlumatlarını normallaşdırmaq ehtiyacını aradan qaldıra bilər). Üstəlik, icra zamanı cəza var: hər bir qatda tələb olunan əlavə hesablamalar səbəbindən neyron şəbəkə proqnozları daha yavaş edir. Xoşbəxtlikdən, təlimdən sonra BN qatını əvvəlki qatla birləşdirmək çox vaxt mümkündür, bununla da icra cəzasından qaçmaq olar. Bu, əvvəlki qatın çəkirlərini və bias-larını yeniləyərək, onun birbaşa müvafiq miqyas və offset ilə çıxışlar istehsal etməsini təmin etməklə edilir. Məsələn, əgər əvvəlki qat $XW + b$ hesablayırsa, BN qatı $\gamma \otimes (XW + b - \mu) / \sigma + \beta$ hesablayacaq (məxrəcdə ϵ hamarlaşdırma terminini nəzərə almadan). Əgər $W' = \gamma \otimes W / \sigma$ və $b' = \gamma \otimes (b - \mu) / \sigma + \beta$ təyin etsək, tənlik $XW' + b'$ -yə sadələşir. Beləliklə, əvvəlki qatın çəkirlərini və bias-larını (W və b) yenilənmiş çəkilər və bias-larla (W' və b') əvəz etsək, BN qatından qurtula bilərik (TFLite çeviricisi bunu avtomatik edir; bax Fəsil 19).

QEYD

Təlimin olduqca yavaş olduğunu görə bilərsiniz, çünki batch normalization istifadə edildikdə hər bir dövr daha çox vaxt alır. Bu, adətən BN ilə yığılmanın daha sürətli olması ilə kompensasiya olunur, buna görə eyni performans çatmaq üçün daha az dövr lazımdır. Ümumilikdə, *wall time* (divar saati ilə ölçülən vaxt) adətən daha qısa olacaq.

Batch Normalization-un Keras-da tətbiqi

Keras-da əksər şeylər kimi, batch normalization tətbiqi sadə və intuitivdir. Sadəcə hər bir gizli qatın aktivasiya funksiyasından əvvəl və ya sonra `BatchNormalization` qatını əlavə edin. Siz həmçinin modelin ilk qatı kimi BN qatını əlavə edə bilərsiniz, lakin bu yerdə adi `Normalization` qatı da eyni dərəcədə yaxşı işləyir (onun yeganə çatışmazlığı əvvəlcə `adapt()` metodunu çağırmalı olmanızdır). Məsələn, bu model hər bir gizli qatdan sonra BN tətbiq edir və modelin ilk qatı kimi (giriş şəkillərini düzləşdirdikdən sonra) BN istifadə edir:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300, activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu",
        kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
```

```
tf.keras.layers.Dense(10, activation="softmax")
])
```

Hamısı budur! Bu kiçik nümunədə cəmi iki gizli qatla batch normalization-un böyük təsiri olmayacaq, lakin daha dərin şəbəkələr üçün böyük fərq yarada bilər.

Model xülasəsini göstərək:

```
>>> model.summary()
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
batch_normalization (Batch Normalization)	(None, 784)	3136
dense (Dense)	(None, 300)	235500
batch_normalization_1 (Batch Normalization)	(None, 300)	1200
dense_1 (Dense)	(None, 100)	30100
batch_normalization_2 (Batch Normalization)	(None, 100)	400
dense_2 (Dense)	(None, 10)	1010

Total params: 271,346
 Trainable params: 268,978
 Non-trainable params: 2,368

Gördüyünüz kimi, hər bir BN qatı hər giriş üçün dörd parametrlə əlavə edir: γ , β , μ və σ (məsələn, birinci BN qatı 3,136 parametrlə əlavə edir, bu 4×784 -dür). Son iki parametrlər, μ və σ , hərəkətli ortalamalardır; onlar backpropagation-dan təsirlənmir, buna görə Keras onları "non-trainable" adlandırır (əgər BN parametrlərinin ümumi sayını saysanız, $3,136 + 1,200 + 400$, və 2-yə bölərsiniz, 2,368 alırsınız, bu modeldəki ümumi non-trainable parametrlərin sayıdır).

Birinci BN qatının parametrlərinə baxaq. İkisi trainable (backpropagation ilə), ikisi isə deyil:

```
>>> [(var.name, var.trainable) for var in model.layers[1].variables]
```

```
[('batch_normalization/gamma:0', True),
 ('batch_normalization/beta:0', True),
 ('batch_normalization/moving_mean:0', False),

 ('batch_normalization/moving_variance:0', False)]
```

BN məqaləsinin müəllifləri BN qatlarını aktivasiya funksiyalarından əvvəl əlavə etməyi müdafiə etdilər (biz indi etdiyimiz kimi sonra yox). Bu barədə bəzi mübahisələr var, çünki hansının üstün olması tapşırıqdan asılı görünür – siz də dataset-inizdə hansı variantın daha yaxşı işlədiyini görmək üçün sınaq keçirə bilərsiniz. BN qatlarını aktivasiya funksiyasından əvvəl əlavə etmək üçün, aktivasiya funksiyalarını gizli qatlardan çıxarmalı və onları BN qatlarından sonra ayrı qatlar kimi əlavə etməlisiniz. Üstəlik, batch normalization qatı hər giriş üçün bir offset parametri ehtiva etdiyi üçün, əvvəlki qatı yaradarkən `use_bias=False` ötürməklə bias terminini çıxara bilərsiniz. Nəhayət, adətən birinci BN qatını atmaq olar, beləliklə birinci gizli qat iki BN qatı arasında sıxılmaz. Yenilənmiş kod belə görünür:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, kernel_initializer="he_normal",
use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(100, kernel_initializer="he_normal",
use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

`BatchNormalization` sinfi tənzimləyə biləcəyiniz bir neçə hiperparametrə malikdir. Defoltlar adətən yaxşı olacaq, lakin bəzən `momentum`-u tənzimləmək lazım ola bilər. Bu hiperparametr `BatchNormalization` qatı tərəfindən eksponensial hərəkətli ortalamaları yenilədikdə istifadə olunur; yeni bir dəyər (yəni, cari batch üzərində hesablanmış giriş ortalamaları və ya standart sapmaları vektoru) v verildikdə, qat işlək ortalama $\hat{v}$ -ni aşağıdakı tənlik ilə yeniləyir:

$$\hat{v} \leftarrow \hat{v} \times \text{momentum} + v \times (1 - \text{momentum})$$

Yaxşı bir `momentum` dəyəri adətən 1-ə yaxındır; məsələn, 0.9, 0.99 və ya 0.999. Daha böyük dataset-lər və daha kiçik mini-batch-lər üçün daha çox 9 istəyirsiniz.

Digər vacib hiperparametr `axis`-dir: o, hansı oxun normallaşdırılacağını müəyyən edir. Defolt olaraq –1-dir, yəni defolt olaraq son oxu normallaşdıracaq (digər oxlar

üzərində hesablanmış ortalama və standart sapmalardan istifadə edərək). Giriş batch-i 2D olduqda (yəni batch forması [batch size, features]), bu o deməkdir ki, hər bir giriş xüsusiyyəti batch-dəki bütün instanslar üzərində hesablanmış ortalama və standart sapma əsasında normallaşdırılacaq. Məsələn, əvvəlki kod nümunəsindəki birinci BN qatı 784 giriş xüsusiyyətinin hər birini müstəqil olaraq normallaşdıracaq (və yenidən miqyaslandıracaq və sürüşdürəcək). Əgər birinci BN qatını Flatten qatından əvvəl yerləşdirsək, giriş batch-ləri 3D olacaq, forması [batch size, height, width]; buna görə də BN qatı 28 ortalama və 28 standart sapma hesablayacaq (hər piksel sütunu üçün 1, batch-dəki bütün instanslar və sütundakı bütün sətirlər üzərində hesablanır) və verilmiş sütundakı bütün pikselləri eyni ortalama və standart sapmadan istifadə edərək normallaşdıracaq. Həmçinin yalnız 28 miqyas parametri və 28 sürüşdürmə parametri olacaq. Əgər əvəzində hər 784 pikseli müstəqil olaraq nəzərə almaq istəyirsinizsə, `axis=[1, 2]` təyin etməlisiniz.

Batch normalization dərin neyron şəbəkələrində, xüsusən də Fəsil 14-də müzakirə ediləcək dərin convolutional neural networks-lərdə ən çox istifadə olunan qatlardan birinə çevrilmişdir, hətta arxitektura diaqramlarında tez-tez buraxılır: BN-nin hər qatdan sonra əlavə edildiyi fərz edilir. İndi təlim zamanı qradiyentləri sabitləşdirmək üçün son bir texnikaya baxaq: gradient clipping.

Gradient Clipping

Exploding gradients problemini yüngülləşdirmək üçün başqa bir texnika, backpropagation zamanı qradiyentləri heç vaxt müəyyən bir həddi aşmamaq üçün kəsməkdir. Buna *gradient clipping* deyilir. Bu texnika ümumiyyətlə recurrent neural networks-lərdə istifadə olunur, çünki batch normalization-un tətbiqi çətinidir (bax Fəsil 15).

Keras-da gradient clipping tətbiq etmək, optimizator yaradarkən `clipvalue` və ya `clipnorm` argumentini təyin etmək qədər sadədir:

```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)

model.compile(..., optimizer=optimizer)
```

Bu optimizator qradiyent vektorunun hər bir komponentini -1.0 ilə 1.0 arasında dəyərə kəsəcək. Bu o deməkdir ki, itkinin (hər bir trainable parametreyə görə) bütün qismən törəmələri -1.0 ilə 1.0 arasında kəsiləcək. Həddi (threshold) tənzimləyə biləcəyiniz bir hiperparametrdir. Qeyd edək ki, bu, qradiyent vektorunun istiqamətini dəyişə bilər. Məsələn, orijinal qradiyent vektoru $[0.9, 100.0]$ olarsa, o, əsasən ikinci ox istiqamətindədir; lakin onu dəyərə görə kəsəndə $[0.9, 1.0]$ alırsınız, bu da təxminən iki ox arasında diaqonala işarə edir. Praktikada bu yanaşma yaxşı işləyir. Əgər gradient

clipping-in qradient vektorunun istiqamətini dəyişmədiyinə əmin olmaq istəyirsinizsə, `clipvalue` əvəzinə `clipnorm` təyin etməklə normaya görə kəsməlisiniz. Bu, əgər vektorun ℓ_2 norması seçdiyiniz həddən böyükdürsə, bütün qradienti kəsəcək. Məsələn, `clipnorm=1.0` təyin etsəniz, $[0.9, 100.0]$ vektoru $[0.00899964, 0.9999595]$ kəsiləcək, istiqaməti qorunacaq, lakin birinci komponent demək olar ki, aradan qaldırılacaq. Əgər təlim zamanı qradientlərin partladığını müşahidə etsəniz (TensorBoard istifadə edərək qradientlərin ölçüsünü izləyə bilərsiniz), müxtəlif həddlərlə dəyər və ya norma üzrə kəsməni sınamaq və validasiya setində ən yaxşı performans hansı variantın verdiyini görmək istəyə bilərsiniz.

Əvvəlcədən Öyrədilmiş Qatların Təkrar İstifadəsi

Çox böyük bir DNN-i sıfırdan öyrətmək, əvvəlcə həll etməyə çalışdığınız tapşırıqə bənzər bir tapşırığı yerinə yetirən mövcud bir neyron şəbəkəsi tapmadan cəhd etmək adətən yaxşı fikir deyil (onları necə tapacağınızı Fəsil 14-də müzakirə edəcəyəm). Əgər belə bir neyron şəbəkəsi tapsanız, ümumiyyətlə onun qatlarının çoxunu, yuxarı qatlar istisna olmaqla, təkrar istifadə edə bilərsiniz. Bu texnika *transfer learning* adlanır. Bu, təlimi əhəmiyyətli dərəcədə sürətləndirməklə yanaşı, əhəmiyyətli dərəcədə daha az təlim məlumatı tələb edir.

Tutaq ki, şəkilləri 100 müxtəlif kateqoriyaya, o cümlədən heyvanlar, bitkilər, nəqliyyat vasitələri və gündəlik əşyalara təsnif etmək üçün öyrədilmiş bir DNN-ə çıxışınız var və indi müəyyən nəqliyyat vasitələrini təsnif etmək üçün bir DNN öyrətmək istəyirsiniz. Bu tapşırıqlar çox oxşardır, hətta qismən üst-üstə düşür, buna görə ilk şəbəkənin hissələrini təkrar istifadə etməyə çalışmalısınız (bax Şəkil 11-5).

QEYD

Əgər yeni tapşırığınız üçün giriş şəkilləri orijinal tapşırıqda istifadə olunanlarla eyni ölçüdə deyilsə, adətən orijinal modelin gözlədiyi ölçüyə uyğunlaşdırmaq üçün preprocessing addımı əlavə etməli olacaqsınız. Ümumiyyətlə, transfer learning girişlərin oxşar aşağı səviyyəli xüsusiyyətlərə malik olduğu zaman daha yaxşı işləyəcək.

Şəkil 11-5. Əvvəlcədən öyrədilmiş qatların təkrar istifadəsi

Orijinal modelin çıxış qatı adətən əvəz edilməlidir, çünki yeni tapşırıq üçün çox güman ki, faydasızdır və yaqin ki, düzgün sayda çıxışa malik olmayacaq.

Eynilə, orijinal modelin yuxarı gizli qatları aşağı qatlar qədər faydalı olmaya bilər, çünki yeni tapşırıq üçün ən faydalı olan yüksək səviyyəli xüsusiyyətlər orijinal tapşırıq

üçün ən faydalı olanlardan əhəmiyyətli dərəcədə fərqlənə bilər. Təkrar istifadə edəcəyiniz düzgün qat sayını tapmaq istəyirsiniz.

İPUCU

Tapşırıqlar nə qədər oxşardırsa, bir o qədər çox qatı təkrar istifadə etmək istəyəcəksiniz (aşağı qatlardan başlayaraq). Çox oxşar tapşırıqlar üçün bütün gizli qatları saxlamağa və yalnız çıxış qatını əvəz etməyə çalışın.

Əvvəlcə bütün təkrar istifadə edilmiş qatları dondurun (yəni, onların çəkirlərini trainable etməyin ki, gradient descent onları dəyişdirməsin və sabit qalsınlar), sonra modelinizi öyrədin və necə işlədiyinə baxın. Sonra yuxarı gizli qatlardan bir-ikisini dondurmaqdan çıxararaq backpropagation-un onları tənzimləməsinə icazə verin və performansın yaxşılaşıb-yaxşılaşmadığına baxın. Nə qədər çox təlim məlumatınız varsa, bir o qədər çox qatı dondurmaqdan çıxara bilərsiniz. Təkrar istifadə edilmiş qatları dondurmaqdan çıxardıqda öyrənmə sürətini azaltmaq da faydalıdır: bu, onların incə tənzimlənmiş çəkirlərini zədələməməyə kömək edəcək.

Əgər hələ də yaxşı performans əldə edə bilmirsinizsə və az təlim məlumatınız varsa, yuxarı gizli qat(lar)ı atmağa və qalan bütün gizli qatları yenidən dondurmağa çalışın. Düzgün təkrar istifadə ediləcək qat sayını tapana qədər iterasiya edə bilərsiniz. Əgər bol təlim məlumatınız varsa, yuxarı gizli qatları atmaq əvəzinə əvəz etməyə və hətta daha çox gizli qat əlavə etməyə cəhd edə bilərsiniz.

Transfer Learning Keras-da

Bir nümunəyə baxaq. Tutaq ki, Fashion MNIST dataset-i yalnız səkkiz sinifdən ibarətdir – məsələn, sandalet və köynək istisna olmaqla bütün siniflər. Kimsə bu set üzərində bir Keras modeli qurub öyrətmiş və kifayət qədər yaxşı performans (>90% dəqiqlik) əldə etmişdir. Gəlin bu modeli A adlandıraq. Siz indi fərqli bir tapşırıqla məşğul olmaq istəyirsiniz: T-shirt və pullover şəkilləriniz var və ikili təsnifatçı öyrətmək istəyirsiniz: T-shirt (və üst geyimlər) üçün müsbət, sandaletlər üçün mənfi. Dataset-iniz olduqca kiçikdir; cəmi 200 etiketli şəkiliniz var. Bu tapşırıq üçün yeni bir model (gəlin onu B modeli adlandıraq) A modeli ilə eyni arxitektura öyrətdikdə, 91.85% test dəqiqliyi alırsınız. Səhər qəhvənizi içərkən, tapşırığınızın A tapşırığına olduqca bənzədiyini başa düşürsünüz, bəlkə transfer learning kömək edə bilər? Gəlin öyrənək!

Əvvəlcə, A modelini yükləməli və onun qatlarına əsaslanan yeni bir model yaratmalısınız. Çıxış qatı istisna olmaqla bütün qatları təkrar istifadə etmək qərarına gəlirsiniz:

```
[...] # Assuming model A was already trained and saved to "my_model_A"
model_A = tf.keras.models.load_model("my_model_A")
model_B_on_A = tf.keras.Sequential(model_A.layers[:-1])

model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Qeyd edək ki, `model_A` və `model_B_on_A` indi bəzi qatları paylaşır. `model_B_on_A`-nı öyrətdikdə, bu, `model_A`-ya da təsir edəcək. Bunun qarşısını almaq istəyirsinizsə, qatlarını təkrar istifadə etməzdən əvvəl `model_A`-nı *clone* etməlisiniz. Bunu etmək üçün `clone_model()` ilə `model_A`-nın arxitekturasını klonlayın, sonra onun çəkirlərini köçürün:

```
model_A_clone = tf.keras.models.clone_model(model_A)

model_A_clone.set_weights(model_A.get_weights())
```

XƏBƏRDARLIQ

`tf.keras.models.clone_model()` yalnız arxitekturanı klonlayır, çəkirləri deyil. Onları `set_weights()` ilə əl ilə köçürməsəniz, klonlanmış model ilk dəfə istifadə edildikdə təsadüfi ilkinləşdiriləcəklər.

İndi B tapşırığı üçün `model_B_on_A`-nı öyrədə bilərsiniz, lakin yeni çıxış qatı təsadüfi ilkinləşdiriləndiyi üçün böyük səhvlər edəcək (heç olmasa ilk bir neçə dövrdə), beləliklə böyük səhv qradientləri təkrar istifadə edilmiş çəkirləri zədələyə bilər. Bunun qarşısını almaq üçün bir yanaşma, ilk bir neçə dövrdə təkrar istifadə edilmiş qatları dondurmaq, yeni qata əqləbatan çəkirlər öyrənmək üçün vaxt verməkdir. Bunu etmək üçün hər bir qatın `trainable` atributunu `False` təyin edin və modeli kompilyasiya edin:

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)

model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,
metrics=["accuracy"])
```

QEYD

Qatları dondurduqdan və ya dondurmaqdan çıxardıqdan sonra modeli həmişə kompilyasiya etməlisiniz.

İndi modeli bir neçə dövr ərzində öyrədə bilərsiniz, sonra təkrar istifadə edilmiş qatları dondurmaqdan çıxarın (bu, modelin yenidən kompilyasiya edilməsini tələb edir) və təkrar istifadə edilmiş qatları B tapşırığı üçün incə tənzimləmək üçün təlimə davam edin. Qatları dondurmaqdan çıxardıqdan sonra, yenidən təkrar istifadə edilmiş çəkirləri zədələməmək üçün adətən öyrənmə sürətini azaltmaq yaxşı fikirdir.

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,  
validation_data=(X_valid_B, y_valid_B))
```

```
for layer in model_B_on_A.layers[:-1]:  
    layer.trainable = True
```

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)  
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,  
metrics=["accuracy"])
```

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,  
validation_data=(X_valid_B, y_valid_B))
```

Bəs son nəticə? Yaxşı, bu modelin test dəqiqliyi 93.85%-dir, 91.85%-dən tam iki faiz bəndi yuxarı! Bu o deməkdir ki, transfer learning səhv nisbətini demək olar ki, 25% azaltdı:

```
>>> model_B_on_A.evaluate(X_test_B, y_test_B)
```

```
[0.2546142041683197, 0.9384999871253967]
```

İnandınız mı? İnanmamalısınız: Mən aldatdım! Güclü təkmilləşdirməni nümayiş etdirən birini tapana qədər çoxlu konfigurasiyaları sınađım. Əgər sinifləri və ya təsadüfi toxumu (random seed) dəyişdirsəniz, təkmilləşdirmənin ümumiyyətlə azaldığını, hətta yox olduğunu və ya tərsinə çevrildiyini görəcəksiniz. Mənim etdiyim "məlumatları etiraf edənə qədər işgəncə vermək" adlanır. Bir məqalə çox müsbət göründükdə, şübhəli olmalısınız: bəlkə parlaq yeni texnika əslində çox kömək etmir (hətta performans pisləşdirə bilər), lakin müəlliflər çoxsaylı variantları sınađılar və yalnız ən yaxşı nəticələri bildirdilər (bu, sadəcə şans ola bilər), yolda nə qədər uğursuzluqla qarşılaşdıqlarını qeyd etmədən. Çox vaxt bu, heç də pis niyyət deyil, lakin elmdə bu qədər çox nəticənin təkrar istehsal edilə bilməməsinin səbəblərindən biridir.

Niyə aldatdım? Məlum olur ki, transfer learning kiçik sıx şəbəkələrlə çox yaxşı işləmir, ehtimal ki, kiçik şəbəkələr az naxış öyrənir və sıx şəbəkələr çox spesifik naxışlar öyrənir, bunlar digər tapşırıqlarda faydalı olma ehtimalı azdır. Transfer learning ən yaxşı dərin convolutional neural networks ilə işləyir, çünki onlar daha ümumi olan feature detectors öyrənməyə meyllidirlər (xüsusən aşağı qatlarda). Biz Fəsil 14-də

transfer learning-ə qayıdacağın, indi müzakirə etdiyimiz texnikalardan istifadə edərək (və bu dəfə aldadılmayacaq, söz verirəm!).

Unsupervised Pretraining

Tutaq ki, mürəkkəb bir tapşırığı həll etmək istəyirsiniz, lakin çox az etiketli təlim məlumatınız var və təəssüf ki, oxşar tapşırıq üzərində öyrədilmiş bir model tapa bilmirsiniz. Ümidinizi itirməyin! Əvvəlcə daha çox etiketli təlim məlumatı toplamağa çalışmalısınız, lakin əgər mümkün deyilsə, *unsupervised pretraining* (nəzarətsiz əvvəlcədən öyrətmə) edə bilərsiniz (bax Şəkil 11-6). Həqiqətən də, etiketsiz təlim nümunələri toplamaq çox vaxt ucuzdur, lakin onları etiketləmək bahalıdır. Əgər çoxlu etiketsiz təlim məlumatı toplaya bilsəniz, onu autoencoder və ya generative adversarial network (GAN; bax Fəsil 17) kimi nəzarətsiz bir model öyrətmək üçün istifadə etməyə cəhd edə bilərsiniz. Sonra autoencoder-in aşağı qatlarını və ya GAN diskriminatorunun aşağı qatlarını təkrar istifadə edərək, üstünə tapşırığınız üçün çıxış qatını əlavə edə və son şəbəkəni supervised learning (yəni, etiketli təlim nümunələri ilə) istifadə edərək incə tənzimləyə bilərsiniz.

Bu texnika Geoffrey Hinton və onun komandası tərəfindən 2006-cı ildə istifadə edilmişdir və neyron şəbəkələrin canlanmasına və deep learning-in uğuruna səbəb olmuşdur. 2010-cu ilə qədər unsupervised pretraining – adətən restricted Boltzmann machines (RBMs; <https://hml.info/extra-anns> saytındakı notebook-a baxın) ilə – dərin şəbəkələr üçün standart idi və yalnız vanishing gradients problemi yüngülləşdirildikdən sonra DNN-ləri tamamilə supervised learning ilə öyrətmək daha geniş yayıldı. Unsupervised pretraining (bu gün adətən RBMs əvəzinə autoencoders və ya GANs istifadə olunur) hələ də mürəkkəb bir tapşırığı həll etməli olduğunuz, təkrar istifadə edə biləcəyiniz oxşar model olmadıqda və az etiketli təlim məlumatı, lakin çoxlu etiketsiz təlim məlumatı olduqda yaxşı bir seçimdir.

Qeyd edək ki, deep learning-in ilk günlərində dərin modelləri öyrətmək çətin idi, buna görə insanlar *greedy layer-wise pretraining* (acgöz qat-qat əvvəlcədən öyrətmə) adlanan bir texnikadan istifadə edirdilər (Şəkil 11-6-da təsvir edilmişdir). Onlar əvvəlcə tək qatlı, adətən RBM olan bir nəzarətsiz model öyrədər, sonra həmin qatı dondurar və onun üzərinə başqa bir qat əlavə edər, sonra modeli yenidən öyrədirdilər (effektiv olaraq yalnız yeni qatı öyrətməklə), sonra yeni qatı dondurar və üzərinə başqa bir qat əlavə edər, modeli yenidən öyrədirdilər və s. İndiki vaxtda işlər daha sadədir: insanlar ümumiyyətlə tam nəzarətsiz modeli bir dəfəyə öyrədirlər və RBMs əvəzinə autoencoders və ya GANs istifadə edirlər.

Şəkil 11-6. Unsupervised training-də model bütün məlumatlar, o cümlədən etiketsiz məlumatlar üzərində nəzarətsiz öyrənmə texnikası ilə öyrədilir, sonra yalnız etiketli məlumatlar üzərində supervised learning texnikası ilə son tapşırıq üçün incə

tənzimlənir; nəzarətsiz hissə şəkildə göstərildiyi kimi bir qatı bir anda öyrədə bilər və ya tam modeli birbaşa öyrədə bilər

Köməkçi Tapşırıqda Pretraining

Əgər çox az etiketli təlim məlumatınız varsa, son bir seçim, asanlıqla əldə edə və ya etiketli təlim məlumatı yarada biləcəyiniz köməkçi tapşırıq üzərində birinci neyron şəbəkəsi öyrətmək, sonra həmin şəbəkənin aşağı qatlarını faktiki tapşırığınız üçün təkrar istifadə etməkdir. Birinci neyron şəbəkəsinin aşağı qatları ikinci neyron şəbəkəsi tərəfindən təkrar istifadə edilə biləcək feature detectors öyrənəcəkdir.

Məsələn, əgər siz üzləri tanıyan bir sistem qurmaq istəyirsinizsə, hər bir fərdin yalnız bir neçə şəklinə sahib ola bilərsiniz – bu, yaxşı bir təsnifatçı öyrətmək üçün açıq-aydın kifayət deyil. Hər bir insanın yüzlərlə şəklini toplamaq praktik olmazdı. Bununla belə, siz internetdə təsadüfi insanların çoxlu şəkillərini toplaya və iki fərqli şəklin eyni insanı əks etdirib-etdirmədiyini aşkar etmək üçün birinci neyron şəbəkəsini öyrədə bilərsiniz. Belə bir şəbəkə üzlər üçün yaxşı feature detectors öyrənəcəkdir, buna görə onun aşağı qatlarını təkrar istifadə etmək az təlim məlumatından istifadə edən yaxşı bir üz təsnifatçısı öyrətməyə imkan verəcəkdir.

Təbii dil emalı (NLP) tətbiqləri üçün, milyonlarla mətn sənədindən ibarət korpus yükləyə və ondan avtomatik olaraq etiketli məlumat yarada bilərsiniz. Məsələn, təsadüfi olaraq bəzi sözləri maskalaya və modeli çatışmayan sözlərin nə olduğunu proqnozlaşdırmaq üçün öyrədə bilərsiniz (məsələn, "What ___ you saying?" cümləsində çatışmayan sözün çox güman ki, "are" və ya "were" olduğunu proqnozlaşdırmalıdır). Əgər modeli bu tapşırıqda yaxşı performansla çatdırarsanız, o, dil haqqında artıq kifayət qədər çox şey biləcək və onu şübhəsiz ki, faktiki tapşırığınız üçün təkrar istifadə edə və etiketli məlumatlarınız üzərində incə tənzimləyə bilərsiniz (Fəsil 15-də daha çox pretraining tapşırıqları müzakirə edəcəyik).

QEYD

Self-supervised learning, siz məlumatın özündən etiketləri avtomatik olaraq yaratdığınız zaman (mətn-maskalama nümunəsində olduğu kimi), sonra nəticədə yaranan "etiketlənmiş" dataset üzərində supervised learning texnikalarından istifadə edərək model öyrətdiyiniz zaman baş verir.

Daha Sürətli Optimizatorlar

Çox böyük bir dərin neyron şəbəkəsini öyrətmək ağırlı dərəcədə yavaş ola bilər. İndiyə qədər təlimi sürətləndirmək (və daha yaxşı həllə çatmaq) üçün dörd üsul gördük: birləşmə çəkilişi üçün yaxşı ilkinləşdirmə strategiyası tətbiq etmək, yaxşı aktivasiya funksiyası istifadə etmək, batch normalization istifadə etmək və əvvəlcədən öyrədilmiş şəbəkənin hissələrini təkrar istifadə etmək (bəlkə də köməkçi tapşırıq üçün və ya unsupervised learning istifadə edərək qurulmuş). Başqa bir böyük sürət artımı adi gradient descent optimizatorundan daha sürətli bir optimizator istifadə etməklə gəlir. Bu bölmədə ən populyar optimallaşdırma alqoritmlərini təqdim edəcəyik: momentum, Nesterov accelerated gradient, AdaGrad, RMSProp və nəhayət Adam və onun variantları.

Momentum

Düz bir səthdə yumşaq yamacla yuvarlanan bir bouling topu təsəvvür edin: o, yavaş başlayacaq, lakin tez bir zamanda momentum qazanacaq və nəhayət terminal sürətinə çatacaq (əgər müəyyən sürtünmə və ya hava müqaviməti varsa). Bu, 1964-cü ildə Boris Polyak tərəfindən təklif edilən *momentum optimization* (momentum optimallaşdırması) fikrinin əsasını təşkil edir. Əksinə, adi gradient descent yumşaq olduqda kiçik addımlar, dik olduqda böyük addımlar atacaq, lakin heç vaxt sürət qazanmayacaq. Nəticədə, adi gradient descent ümumiyyətlə minimuma çatmaq üçün momentum optimallaşdırmasından çox daha yavaştır.

Xatırlayın ki, gradient descent çəkilişi θ -nı birbaşa olaraq məsrəf funksiyasının $J(\theta)$ çəkilişə görə qradiyentini ($\nabla \theta J(\theta)$) öyrənmə sürəti η ilə vurub çıxmaqla yeniləyir. Tənlik $\theta \leftarrow \theta - \eta \nabla \theta J(\theta)$ -dir. O, əvvəlki qradiyentlərin nə olduğu ilə maraqlanmır. Əgər yerli qradiyent çox kiçikdirsə, o, çox yavaş gedir.

Momentum optimallaşdırması əvvəlki qradiyentlərin nə olduğuna çox əhəmiyyət verir: hər bir iterasiyada o, yerli qradiyenti *momentum vector* m -dən (öyrənmə sürəti η ilə vurulmuş) çıxarır və bu momentum vektorunu əlavə etməklə çəkilişi yeniləyir (bax Tənlik 11-5). Başqa sözlə, qradiyent sürət kimi deyil, təcil kimi istifadə olunur. Bir növ sürtünmə mexanizmini simulyasiya etmək və momentumun çox böyüməsinin qarşısını almaq üçün alqoritm β adlı yeni bir hiperparametr təqdim edir, buna *momentum* deyilir və 0 (yüksək sürtünmə) ilə 1 (sürtünməsiz) arasında təyin edilməlidir. Tipik bir momentum dəyəri 0.9-dur.

Tənlik 11-5. Momentum alqoritmi

1. $m \leftarrow \beta m - \eta \nabla \theta J(\theta)$
2. $\theta \leftarrow \theta + m$

Əgər qradient sabit qalırsa, terminal sürətinin (yəni, çəki yeniləmələrinin maksimum ölçüsü) həmin qradient ilə öyrənmə sürəti η və $1/(1 - \beta)$ -nin hasilinə bərabər olduğunu yoxlaya bilərsiniz (işarəni nəzərə almadan). Məsələn, $\beta = 0.9$ olarsa, terminal sürəti qradient dəfə öyrənmə sürətinin 10 qatına bərabərdir, beləliklə momentum optimallaşdırması gradient descent-dən 10 dəfə daha sürətli gedir! Bu, momentum optimallaşdırmasının gradient descent-dən daha sürətli plato (plateaus) dan qaçmasına imkan verir. Fəsil 4-də gördük ki, girişlər çox fərqli miqyasa malik olduqda, məsrəf funksiyası uzanmış bir qab kimi görünəcək (bax Şəkil 4-7). Gradient descent dik yamacdan olduqca sürətli düşür, lakin sonra dəre (valley) boyunca aşağı düşmək üçün çox uzun vaxt tələb edir. Əksinə, momentum optimallaşdırması dəre boyunca getdikcə daha sürətli yuvarlanacaq, nəhayət dibə (optimala) çatacaq. Batch normalization istifadə etməyən dərin neyron şəbəkələrdə, yuxarı qatlar çox vaxt çox fərqli miqyaslı girişlərə sahib olacaq, buna görə momentum optimallaşdırması çox kömək edir. O, həmçinin yerli optimallaşdırmalardan yuvarlanmağa kömək edə bilər.

QEYD

Momentum səbəbindən optimizator bir qədər aşağı düşür, sonra geri qayıdaraq, yenidən aşağı düşür və minimumda sabitləşməzdən əvvəl bir neçə dəfə bu şəkildə salınım edə bilər. Bu, sistemdə bir az sürtünməyə sahib olmağın yaxşı olmasının səbəblərindən biridir: bu, salınımları aradan qaldırır və beləliklə yığılmanı sürətləndirir.

Momentum optimallaşdırmasını Keras-da tətbiq etmək heç bir çətinlik tələb etmir: sadəcə SGD optimizatorundan istifadə edin və onun `momentum` hiperparametrini təyin edin, sonra arxaya yaslanın və faydalanın!

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)
```

Momentum optimallaşdırmasının bir çatışmazlığı, tənzimləmək üçün daha bir hiperparametr əlavə etməsidir. Bununla belə, 0.9 momentum dəyəri adətən praktikada yaxşı işləyir və demək olar ki, həmişə adi gradient descent-dən daha sürətli olur.

Nesterov Accelerated Gradient

Momentum optimallaşdırmasının kiçik bir variantı, 1983-cü ildə Yurii Nesterov tərəfindən təklif edilmişdir, demək olar ki, həmişə adi momentum optimallaşdırmasından daha sürətlidir. *Nesterov accelerated gradient* (NAG) metodu, həmçinin *Nesterov momentum optimization* kimi tanınır, məsrəf funksiyasının

qradiyentini yerli mövqedə θ deyil, momentum istiqamətində bir az irəlində, $\theta + m\beta$ nöqtəsində ölçür (bax Tənlik 11-6).

Tənlik 11-6. Nesterov accelerated gradient alqoritmi

1. $m \leftarrow \beta m - \eta \nabla_{\theta} J(\theta + \beta m)$
2. $\theta \leftarrow \theta + m$

Bu kiçik dəyişiklik işləyir, çünki ümumiyyətlə momentum vektoru düzgün istiqamətdə (yəni optimala doğru) işarə edəcəkdir, buna görə də orijinal mövqedəki qradiyent əvəzinə həmin istiqamətdə bir az daha uzaqda ölçülən qradiyentdən istifadə etmək bir qədər daha dəqiq olacaq, Şəkil 11-7-də görə bilərsiniz (burada ∇_1 başlanğıc nöqtəsi θ -da ölçülən məsrəf funksiyasının qradiyentini, ∇_2 isə $\theta + m\beta$ nöqtəsində ölçülən qradiyenti təmsil edir).

Şəkil 11-7. Adi və Nesterov momentum optimallaşdırması: birincisi momentum addımından əvvəl hesablanan qradiyentləri tətbiq edir, ikincisi isə momentum addımından sonra hesablanan qradiyentləri tətbiq edir

Gördüyünüz kimi, Nesterov yeniləməsi optimala daha yaxın bitir. Bir müddətdən sonra bu kiçik təkmilləşdirmələr toplanır və NAG nəticədə adi momentum optimallaşdırmasından əhəmiyyətli dərəcədə daha sürətli olur. Üstəlik, diqqət yetirin ki, momentum çəkiləri bir dərədən keçirdikdə, ∇_1 dərədən daha da itələməyə davam edir, ∇_2 isə dərənin dibinə doğru geri itələyir. Bu, salınımları azaltmağa kömək edir və beləliklə NAG daha sürətli yığılır.

NAG-dan istifadə etmək üçün, SGD optimizatorunu yaradarkən sadəcə `nesterov=True` təyin edin:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9,
nesterov=True)
```

AdaGrad

Uzanmış qab problemini bir daha nəzərdən keçirək: gradient descent ən dik yamacdan sürətlə düşməklə başlayır, lakin bu, qlobal optimala düz istiqamətdə deyil, sonra dərənin dibinə çox yavaş-yavaş enir. Alqoritmin istiqamətini daha əvvəl düzəldib qlobal optimala bir az daha çox yönəltməsi yaxşı olardı. *AdaGrad* alqoritmi qradiyent vektorunu ən dik ölçülər boyunca miqyasını azaltmaqla bu düzəlişi əldə edir (bax Tənlik 11-7).

Tənlik 11-7. AdaGrad alqoritmi

1. $s \leftarrow s + \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + \epsilon}$

Birinci addım qradientlərin kvadratını s vektorunda toplayır (xatırlayın ki, $\otimes$ simvolu element-wise vurmanı təmsil edir). Bu vektor forması hər bir s elementinin s_i üçün $s_i \leftarrow s_i + (\partial_i J(\theta) / \partial \theta_i)^2$ hesablanmasına ekvivalentdir; yəni hər bir s_i parametrlər θ_i ilə bağlı məsrəflə funksiyasının qismən törəməsinin kvadratlarını toplayır. Əgər məsrəflə funksiyası i ci ölçü boyunca dikdirsə, s_i hər iterasiyada getdikcə daha böyük olacaq.

İkinci addım gradient descent ilə demək olar ki, eynidir, lakin böyük bir fərqlə: qradient vektoru $\sqrt{s + \epsilon}$ amililə miqyaslandırılır ($\oslash$ simvolu element-wise bölməni təmsil edir, ϵ isə sıfıra bölmənin qarşısını almaq üçün hamarlaşdırma terminidir, adətən 10^{-10} təyin edilir). Bu vektor forması bütün parametrlər θ_i üçün eyni vaxtda $\theta_i \leftarrow \theta_i - \eta \partial J(\theta) / \partial \theta_i / \sqrt{s_i + \epsilon}$ hesablanmasına ekvivalentdir.

Qısacası, bu alqoritm öyrənmə sürətini azaldır, lakin bunu yumşaq yamaqlara nisbətən dik ölçülər üçün daha sürətli edir. Buna *adaptive learning rate* (adaptiv öyrənmə sürəti) deyilir. Bu, nəticədə yeniləmələri qlobal optimala daha birbaşa yönəltməyə kömək edir (bax Şəkil 11-8). Digər bir üstünlük, öyrənmə sürəti hiperparametri η -nın daha az tənzimlənməsini tələb etməsidir.

Şəkil 11-8. AdaGrad ilə gradient descent: birincisi istiqamətini optimala yönəltmək üçün daha əvvəl düzəldə bilər

AdaGrad sadə kvadratik problemlər üçün tez-tez yaxşı çıxış edir, lakin neyron şəbəkələri öyrədərkən tez-tez çox tez dayanır: öyrənmə sürəti o qədər azalır ki, alqoritm qlobal optimala çatmadan tamamilə dayanır. Buna görə Keras-da Adagrad optimizatoru olsa da, dərin neyron şəbəkələrini öyrətmək üçün ondan istifadə etməməlisiniz (baxmayaraq ki, linear regression kimi daha sadə tapşırıqlar üçün səmərəli ola bilər). Yenə də, AdaGrad-ı başa düşmək digər adaptiv öyrənmə sürəti optimizatorlarını anlamağa kömək edir.

RMSPProp

Gördüyümüz kimi, AdaGrad bir az çox tez yavaşlamaq və qlobal optimala heç vaxt yığılmamaq riski daşıyır. *RMSPProp* alqoritmi bunu düzəltmək üçün təlimin əvvəlindən bütün qradientlər əvəzinə yalnız ən son iterasiyalardan olan qradientləri toplayır. O, birinci addımda eksponensial azalma istifadə etməklə bunu edir (bax Tənlik 11-8).

Tənlik 11-8. RMSPProp alqoritmi

1. $s \leftarrow \rho s + (1 - \rho) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
2. $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta) \oslash \sqrt{s + \epsilon}$

Çürümə sürəti ρ adətən 0.9 təyin edilir. Bəli, bu yenə yeni bir hiperparametrdir, lakin bu defolt dəyər çox vaxt yaxşı işləyir, buna görə onu ümumiyyətlə tənzimləmək lazım olmaya bilər.

Təxmin etdiyiniz kimi, Keras-da RMSprop optimizatoru var:

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

Çox sadə problemlər istisna olmaqla, bu optimizator demək olar ki, həmişə AdaGrad-dan daha yaxşı çıxış edir. Əslində, Adam optimallaşdırması ortaya çıxana qədər bir çox tədqiqatçıların üstünlük verdiyi optimallaşdırma alqoritmi idi.

Adam

Adam, adaptive moment estimation deməkdir, momentum optimallaşdırması və RMSProp-un fikirlərini birləşdirir: momentum optimallaşdırması kimi, o, keçmiş qradientlərin eksponensial çürüyən ortalamasını izləyir; RMSProp kimi, o, keçmiş kvadrat qradientlərin eksponensial çürüyən ortalamasını izləyir (bax Tənlik 11-9). Bunlar qradientlərin ortalama və (mərkəzləşdirilməmiş) dispersiyasının təxminləridir. Ortalama tez-tez *first moment*, dispersiya isə *second moment* adlanır, buna görə alqoritmin adı.

Tənlik 11-9. Adam alqoritmi

1. $m \leftarrow \beta_1 m - (1 - \beta_1) \nabla_{\theta} J(\theta)$
2. $s \leftarrow \beta_2 s + (1 - \beta_2) \nabla_{\theta} J(\theta) \otimes \nabla_{\theta} J(\theta)$
3. $\hat{m} \leftarrow m / (1 - \beta_1^t)$
4. $\hat{s} \leftarrow s / (1 - \beta_2^t)$
5. $\theta \leftarrow \theta + \eta \hat{m} \oslash \sqrt{\hat{s} + \epsilon}$

Bu tənlikdə t iterasiya nömrəsini təmsil edir (1-dən başlayaraq).

Əgər yalnız 1, 2 və 5-ci addımlara baxsanız, Adam-ın həm momentum optimallaşdırmasına, həm də RMSProp-a yaxın oxşarlığını görərsiniz: β_1 momentum optimallaşdırmasındakı β -ya, β_2 isə RMSProp-dakı ρ -ya uyğundur. Yeganə fərq, 1-ci addımın eksponensial çürüyən cəmi əvəzinə eksponensial çürüyən ortalama hesablamasıdır, lakin bunlar sabit bir amildən başqa ekvivalentdirlər (çürüyən ortalama sadəcə $1 - \beta_1$ dəfə çürüyən cəmidir). 3 və 4-cü addımlar bir qədər texniki detaldır: m və s 0 ilə ilkinləşdirildiyi üçün, təlimin əvvəlində 0-a doğru meyilli olacaqlar, buna görə bu iki addım təlimin əvvəlində m və s -ni gücləndirməyə kömək edəcək.

Momentum çürümə hiperparametri β_1 adətən 0.9-a, miqyas çürümə hiperparametri β_2 isə tez-tez 0.999-a ilkinləşdirilir. Hamarlaşdırma termini ϵ adətən 10^{-7} kimi kiçik bir

ədədə təyin edilir. Bunlar Adam sinfi üçün defolt dəyərlərdir. Keras istifadə edərək Adam optimizatorunu necə yaratmaq olar:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9,
beta_2=0.999)
```

Adam AdaGrad və RMSProp kimi adaptiv öyrənmə sürəti alqoritmi olduğundan, öyrənmə sürəti hiperparametri η -nın daha az tənzimlənməsini tələb edir. Tez-tez defolt dəyər $\eta = 0.001$ istifadə edə bilərsiniz, bu da Adam-ı gradient descent-dən daha asan istifadə edilən edir.

İPUCU

Bütün bu müxtəlif texnikalardan boğulmağa başlayırsınızsa və tapşırığınız üçün doğru olanları necə seçəcəyinizi düşünürsünüzsə, narahat olmayın: bəzi praktik göstərişlər bu fəslin sonunda verilmişdir.

Nəhayət, Adam-ın üç variantını qeyd etmək lazımdır: AdaMax, Nadam və AdamW.

AdaMax

Adam məqaləsi həmçinin AdaMax-ı təqdim etdi. Diqqət yetirin ki, Tənlik 11-9-un 2-ci addımında Adam qradientlərin kvadratlarını s -də toplayır (son qradientlər üçün daha böyük çəki ilə). 5-ci addımda, əgər ϵ və 3-4-cü addımları nəzərə almasaq (hər halda texniki detallardır), Adam parametr yeniləmələrini s -nin kvadrat kökü ilə miqyaslandırır. Qısacası, Adam parametr yeniləmələrini zamanla çürümüş qradientlərin ℓ_2 norması ilə miqyaslandırır (xatırlayın ki, ℓ_2 norması kvadratların cəminin kvadrat köküdür).

AdaMax ℓ_2 normasını ℓ_∞ norması ilə əvəz edir (yəni maksimum). Xüsusilə, Tənlik 11-9-un 2-ci addımını $s \leftarrow \max(\beta_2 s, |\nabla \theta J(\theta)|)$ ilə əvəz edir, 4-cü addımı atır və 5-ci addımda qradient yeniləmələrini s amili ilə miqyaslandırır, bu da zamanla çürümüş qradientlərin mütləq dəyərinin maksimumudur.

Praktikada, bu AdaMax-ı Adam-dan daha sabit edə bilər, lakin bu, həqiqətən dataset-dən asılıdır və ümumiyyətlə Adam daha yaxşı performans göstərir. Beləliklə, bu, sadəcə bəzi tapşırıqlarda Adam ilə problem yaşasanız, sınağa biləcəyiniz daha bir optimizatorudur.

Nadam

Nadam optimallaşdırması Adam optimallaşdırması üstəgəl Nesterov hiyləsidir, buna görə tez-tez Adam-dan bir qədər daha sürətli yığılacaq. Bu texnikanı təqdim edən tədqiqatçı Timothy Dozat müxtəlif tapşırıqlarda çoxsaylı optimizatorları müqayisə edir və Nadam-ın ümumiyyətlə Adam-dan üstün olduğunu, lakin bəzən RMSProp tərəfindən üstünlənə biləcəyini tapır.

AdamW

AdamW, *weight decay* (çəki çürüməsi) adlanan bir regularizasiya texnikasını inteqrasiya edən Adam variantıdır. Weight decay hər bir təlim iterasiyasında modelin çəkirlərinin ölçüsünü 0.99 kimi bir çürümə amili ilə vuraraq azaldır. Bu, Fəsil 4-də təqdim edilən ℓ_2 regularizasiyasını xatırlada bilər, bu da çəkirləri kiçik saxlamağı hədəfləyir və həqiqətən də riyazi olaraq göstərilə bilər ki, SGD istifadə edildikdə ℓ_2 regularizasiyası weight decay-a ekvivalentdir. Lakin Adam və ya onun variantları istifadə edildikdə, ℓ_2 regularizasiyası və weight decay *ekvivalent deyil*: praktikada Adam-ı ℓ_2 regularizasiyası ilə birləşdirmək, tez-tez SGD tərəfindən istehsal olunan modellər qədər yaxşı ümumiləşməyən modellərlə nəticələnir. AdamW, Adam-ı weight decay ilə düzgün birləşdirərək bu problemi düzəldir.

XƏBƏRDARLIQ

Adaptiv optimallaşdırma üsulları (RMSProp, Adam, AdaMax, Nadam və AdamW optimallaşdırması daxil olmaqla) çox vaxt əladır, yaxşı bir həllə sürətlə yığılır. Bununla belə, 2017-ci ildə Ashia C. Wilson və digərlərinin məqaləsi göstərdi ki, onlar bəzi dataset-lərdə zəif ümumiləşən həllərə səbəb ola bilər. Buna görə modelinizin performansından məyus olduqda, NAG istifadə etməyə çalışın: dataset-iniz adaptiv qradientlərə qarşı allergiyası ola bilər. Həmçinin ən son tədqiqatları izləyin, çünki bu sahə sürətlə inkişaf edir.

Keras-da Nadam, AdaMax və ya AdamW istifadə etmək üçün

```
tf.keras.optimizers.Adam-ı tf.keras.optimizers.Nadam,  
tf.keras.optimizers.Adamax və ya tf.keras.optimizers.experimental.AdamW  
ilə əvəz edin. AdamW üçün yaqın ki, weight_decay hiperparametrini tənzimləmək  
istəyəcəksiniz.
```

İndiyə qədər müzakirə edilən bütün optimallaşdırma texnikaları yalnız *birinci dərəcəli qismən törəmələrə* (Jacobians) əsaslanır. Optimallaşdırma ədəbiyyatında *ikinci dərəcəli qismən törəmələrə* (Hessians, yəni Jacobians-ın qismən törəmələri) əsaslanan heyranəmiz alqoritmlər də var. Təəssüf ki, bu alqoritmləri dərin neyron şəbəkələrinə tətbiq etmək çox çətindir, çünki hər çıxış üçün 2^n Hessians var

(burada n parametrlərin sayıdır), əksinə hər çıxış üçün yalnız n Jacobians var. DNN-lər adətən on minlərlə və ya daha çox parametmə malik olduğundan, ikinci dərəcəli optimallaşdırma alqoritmləri tez-tez yaddaşa belə sığmır və hətta sığdıqda belə, Hessians-ı hesablamaq çox yavaşdır.

SEYRƏK MODELƏRİN ÖYRƏDİLMƏSİ

Müzakirə etdiyimiz bütün optimallaşdırma alqoritmləri sıx modellər istehsal edir, yəni parametrlərin əksəriyyəti sıfırdan fərqli olacaq. İcra zamanı ildırım sürəti ilə işləyən bir modelə ehtiyacınız varsa və ya daha az yaddaş tutmasını istəyirsinizsə, əvəzində seyrək bir model əldə etməyə üstünlük verə bilərsiniz.

Buna nail olmağın bir yolu modeli adi qaydada öyrətmək, sonra kiçik çəkildən qurtulmaqdır (onları sıfıra təyin edin). Lakin bu, adətən çox seyrək bir modelə gətirib çıxarmaz və modelin performansını pisləşdirə bilər.

Daha yaxşı bir seçim, təlim zamanı güclü ℓ_1 regularizasiyası tətbiq etməkdir (bu fəsilə sonra necə olduğunu görəcəksiniz), çünki o, optimizatoru mümkün qədər çox çəki sıfırlamağa məcbur edir (Lasso Regression-də müzakirə edildiyi kimi).

Əgər bu texnikalar kifayət etmirsə, TensorFlow Model Optimization Toolkit (TF-MOT) baxın. O, təlim zamanı çəkilərin böyüklüyünə əsaslanaraq əlaqələri iterativ olaraq çıxara bilən bir pruning API təmin edir.

Cədvəl 11-2 indiyə qədər müzakirə etdiyimiz bütün optimizatorları müqayisə edir (* pis, ** orta, *** yaxşı).

Cədvəl 11-2. Optimizatorların müqayisəsi

Sinif	Yığılma sürəti	Yığılma keyfiyyəti
SGD	*	***
SGD(momentum=...)	**	***
SGD(momentum=..., nesterov=True)	**	***
Adagrad	***	* (çox tez dayanır)

RMSprop	***	** və ya ***
Adam	***	** və ya ***
AdaMax	***	** və ya ***
Nadam	***	** və ya ***
AdamW	***	** və ya ***

Öyrənmə Sürətinin Planlaşdırılması

Yaxşı bir öyrənmə sürəti tapmaq çox vacibdir. Əgər onu çox yüksək təyin etsəniz, təlim ayrıla bilər (Gradient Descent bölməsində müzakirə edildiyi kimi). Əgər onu çox aşağı təyin etsəniz, təlim nəhayət optimala yığılacaq, lakin çox uzun vaxt aparacaq. Əgər onu bir az çox yüksək təyin etsəniz, əvvəlcə çox sürətli irəliləyiş edəcək, lakin optimal ətrafında rəqs edəcək və heç vaxt tamamilə sabitləşməyəcək. Əgər məhdud hesablama büdcəniz varsa, təlimi düzgün yığılmadan əvvəl kəsməli ola bilərsiniz, bu da suboptimal həllə gətirib çıxarır (bax Şəkil 11-9).

Şəkil 11-9. Müxtəlif öyrənmə sürətləri η üçün öyrənmə əyriləri

Fəsil 10-da müzakirə edildiyi kimi, modeli bir neçə yüz iterasiya üçün öyrədərək, öyrənmə sürətini çox kiçik bir dəyərdən çox böyük bir dəyərə eksponensial olaraq artıraraq, sonra öyrənmə əyrisinə baxaraq və öyrənmə əyrisinin yenidən yuxarı qalxmağa başladığı dəyərdən bir az aşağı bir öyrənmə sürəti seçməklə yaxşı bir öyrənmə sürəti tapa bilərsiniz. Sonra modelinizi yenidən ilkinləşdirib həmin öyrənmə sürəti ilə öyrədə bilərsiniz.

Lakin sabit bir öyrənmə sürətindən daha yaxşısını edə bilərsiniz: əgər böyük bir öyrənmə sürəti ilə başlayıb, sonra təlim sürətli irəliləyiş etməyi dayandırdıqda onu azaltsanız, optimal sabit öyrənmə sürəti ilə müqayisədə daha sürətli yaxşı bir həllə çata bilərsiniz. Təlim zamanı öyrənmə sürətini azaltmaq üçün çoxlu müxtəlif strategiyalar var. Aşağı öyrənmə sürəti ilə başlamaq, sonra artırmaq, sonra yenidən azaltmaq da faydalı ola bilər. Bu strategiyalar *learning schedules* (öyrənmə planları) adlanır (bu konsepti Fəsil 4-də qısaca təqdim etmişdim). Bunlar ən çox istifadə olunan öyrənmə planlarıdır:

Power scheduling (Qüvvət planlaşdırması)

Öyrənmə sürətini iterasiya nömrəsinin t funksiyası kimi təyin edin: $\eta(t) = \eta_0 / (1 + t / s)c$. İlk öyrənmə sürəti η_0 , qüvvət c (adətən 1 təyin edilir) və addımlar s hiperparametrlərdir. Öyrənmə sürəti hər addımda azalır. s addımdan sonra öyrənmə sürəti $\eta_0 / 2$ -yə düşür. Daha s addımdan sonra $\eta_0 / 3$ -ə, sonra $\eta_0 / 4$ -ə, sonra $\eta_0 / 5$ -ə və s. Gördüyünüz kimi, bu plan əvvəlcə tez, sonra getdikcə daha yavaş azalır. Əlbəttə, power scheduling η_0 və s -nin (və bəlkə də c) tənzimlənməsini tələb edir.

Exponential scheduling (Eksponensial planlaşdırma)

Öyrənmə sürətini $\eta(t) = \eta_0 \cdot 0.1^{\{t/s\}}$ təyin edin. Öyrənmə sürəti tədricən hər s addımda 10 amili ilə azalacaq. Power scheduling öyrənmə sürətini getdikcə daha yavaş azaldarkən, exponential scheduling onu hər s addımda 10 amili ilə kəsməyə davam edir.

Piecewise constant scheduling (Parçalı sabit planlaşdırma)

Bir neçə dövr üçün sabit bir öyrənmə sürəti istifadə edin (məsələn, 5 dövr üçün $\eta_0 = 0.1$), sonra başqa bir neçə dövr üçün daha kiçik bir öyrənmə sürəti (məsələn, 50 dövr üçün $\eta_1 = 0.001$) və s. Bu həll çox yaxşı işləyə bilər, lakin düzgün öyrənmə sürətləri ardıcılığını və hər birini nə qədər müddət istifadə edəcəyinizi anlamaq üçün əngəl tələb edir.

Performance scheduling (Performans planlaşdırması)

Hər N addımda validasiya səhvini ölçün (early stopping-da olduğu kimi) və səhv düşməyi dayandırdıqda öyrənmə sürətini λ amili ilə azaldın.

1cycle scheduling

1cycle 2018-ci ildə Leslie Smith tərəfindən təqdim edilmişdir. Digər yanaşmalardan fərqli olaraq, o, ilkin öyrənmə sürəti η_0 -ı artırmaqla başlayır, təlimin yarısına qədər xətti olaraq η_1 -ə qədər böyüdür. Sonra təlimin ikinci yarısı ərzində öyrənmə sürətini xətti olaraq yenidən η_0 -a endirir və son bir neçə dövrü sürəti bir neçə dərəcə aşağı salaraq (yenə xətti) bitirir. Maksimum öyrənmə sürəti η_1 , optimal öyrənmə sürətini tapmaq üçün istifadə etdiyimiz eyni yanaşma ilə seçilir və ilkin öyrənmə sürəti η_0 adətən 10 dəfə aşağı olur. Momentum istifadə edərkən, əvvəlcə yüksək momentumla başlayırıq (məsələn, 0.95), sonra təlimin birinci yarısında onu daha aşağı bir momentuma endiririk (məsələn, 0.85-ə qədər xətti), sonra təlimin ikinci yarısında onu yenidən maksimum dəyəərə (məsələn, 0.95) qaytarırıq, son bir neçə dövrü bu maksimum dəyərlə bitiririk. Smith çoxsaylı təcrübələr apardı və bu yanaşmanın tez-tez təlimi əhəmiyyətli dərəcədə sürətləndirə və daha yaxşı performansla çata biləcəyini göstərdi. Məsələn, populyar CIFAR10 şəkil dataset-də bu yanaşma cəmi 100 dövrdə 91.9% validasiya dəqiqliyinə çatdı, standart yanaşma ilə 800 dövrdə

90.3% dəqiqliklə müqayisədə (eyni neyron şəbəkə arxitekturası ilə). Bu uğur *superconvergence* adlandırıldı.

2013-cü ildə Andrew Senior və digərlərinin məqaləsi momentum optimallaşdırması ilə dərin neyron şəbəkələrini nitq tanıma üçün öyrədərkən ən populyar öyrənmə planlarının bəzilərinin performansını müqayisə etdi. Müəlliflər belə nəticəyə gəldilər ki, bu şəraitdə həm performance scheduling, həm də exponential scheduling yaxşı işləmişdir. Onlar exponential scheduling-ə üstünlük verdilər, çünki onu tənzimləmək asan idi və optimal həllə bir az daha sürətli yığıldı. Onlar həmçinin qeyd etdilər ki, onu performance scheduling-dən daha asan tətbiq etmək olar, lakin Keras-da hər iki variant asandır. Bununla belə, 1cycle yanaşması daha da yaxşı görünür.

Power scheduling-i Keras-da tətbiq etmək ən asan seçimdir – sadəcə optimizator yaradarkən `decay` hiperparametrini təyin edin:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, decay=1e-4)
```

`decay` s-nin tərsidir (öyrənmə sürətini daha bir vahidə bölmək üçün lazım olan addımların sayı) və Keras c-nin 1-ə bərabər olduğunu fərz edir.

Exponential scheduling və piecewise scheduling olduqca sadədir. Əvvəlcə cari dövrü qəbul edən və öyrənmə sürətini qaytaran bir funksiya müəyyən etməlisiniz. Məsələn, exponential scheduling tətbiq edək:

```
def exponential_decay_fn(epoch):  
  
    return 0.01 * 0.1 ** (epoch / 20)
```

Əgər η_0 və s-ni hardcode etmək istəmirsinizsə, konfigurasiya edilmiş funksiyanı qaytaran bir funksiya yarada bilərsiniz:

```
def exponential_decay(lr0, s):  
    def exponential_decay_fn(epoch):  
        return lr0 * 0.1 ** (epoch / s)  
    return exponential_decay_fn
```

```
exponential_decay_fn = exponential_decay(lr0=0.01, s=20)
```

Sonra, plan funksiyanı verərək `LearningRateScheduler` callback-i yaradın və bu callback-i `fit()` metoduna ötürün:

```
lr_scheduler =  
tf.keras.callbacks.LearningRateScheduler(exponential_decay_fn)
```

```
history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

`LearningRateScheduler` hər dövrün əvvəlində optimizatorun `learning_rate` atributunu yeniləyəcək. Öyrənmə sürətini hər dövrə bir dəfə yeniləmək adətən kifayətdir, lakin əgər onu daha tez-tez, məsələn, hər addımda yeniləmək istəyirsinizsə, həmişə öz callback-inizi yazı bilərsiniz (bu fəslin notebook-unun "Exponential Scheduling" bölməsinə baxın). Hər addımda öyrənmə sürətini yeniləmək, əgər hər dövrdə çoxlu addımlar varsa, kömək edə bilər. Alternativ olaraq, qısa müddətdə təsvir edilən `tf.keras.optimizers.schedules` yanaşmasından istifadə edə bilərsiniz.

İPUCU

Təlimdən sonra `history.history["lr"]` sizə təlim zamanı istifadə edilən öyrənmə sürətlərinin siyahısına çıxış verir.

Plan funksiyası istəyə bağlı olaraq cari öyrənmə sürətini ikinci argument kimi qəbul edə bilər. Məsələn, aşağıdakı plan funksiyası əvvəlki öyrənmə sürətini $0.1^{1/20}$ ilə vurur, bu da eyni eksponensial çürümə ilə nəticələnir (çürümə indi dövr 0-ın əvvəlində başlayır, 1 əvəzinə):

```
def exponential_decay_fn(epoch, lr):  
    return lr * 0.1 ** (1 / 20)
```

Bu tətbiq optimizatorun ilkin öyrənmə sürətinə əsaslanır (əvvəlki tətbiqdən fərqli olaraq), buna görə onu müvafiq olaraq təyin etdiyinizə əmin olun.

Bir modeli saxladığınız zaman, optimizator və onun öyrənmə sürəti onunla birlikdə saxlanılır. Bu o deməkdir ki, bu yeni plan funksiyası ilə, sadəcə öyrədilmiş modeli yükləyə və qaldığı yerdən təlimə davam edə bilərsiniz, heç bir problem yoxdur. Bununla belə, plan funksiyanız `epoch` argumentindən istifadə edərsə, işlər o qədər də sadə deyil: dövr saxlanılmır və `fit()` metodunu hər çağırduğunuzda 0-a sıfırlanır. Əgər modeli qaldığı yerdən təlimə davam etdirsəniz, bu, çox böyük bir öyrənmə sürətinə səbəb ola bilər ki, bu da yaxın ki, modelinizin çəkirlərini zədələyəcək. Bir həll, `fit()` metodunun `initial_epoch` argumentini əl ilə təyin etməkdir ki, dövr düzgün dəyərdən başlasın.

Piecewise constant scheduling üçün, aşağıdakı kimi bir plan funksiyasından istifadə edə bilərsiniz (daha əvvəl olduğu kimi, əgər istəsəniz, daha ümumi bir funksiya təyin edə bilərsiniz; notebook-un "Piecewise Constant Scheduling" bölməsinə baxın), sonra bu funksiya ilə `LearningRateScheduler` callback-i yaradın və exponential scheduling-də olduğu kimi onu `fit()`-ə ötürün:

```
def piecewise_constant_fn(epoch):
    if epoch < 5:
        return 0.01
    elif epoch < 15:
        return 0.005
    else:
        return 0.001
```

Performance scheduling üçün `ReduceLROnPlateau` callback-indən istifadə edin. Məsələn, aşağıdakı callback-i `fit()` metoduna ötürsəniz, ən yaxşı validasiya itkisi ardıcıl beş dövr ərzində yaxşılaşmadıqda, öyrənmə sürətini 0.5-ə vurur (digər seçimlər mövcuddur; daha ətraflı məlumat üçün sənədlərə baxın):

```
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)

history = model.fit(X_train, y_train, [...], callbacks=[lr_scheduler])
```

Nəhayət, Keras öyrənmə sürətinin planlaşdırılmasını tətbiq etmək üçün alternativ bir yol təklif edir: `tf.keras.optimizers.schedules`-də mövcud olan siniflərdən birini istifadə edərək planlaşdırılmış bir öyrənmə sürəti təyin edə və sonra onu istənilən optimizatora ötürə bilərsiniz. Bu yanaşma öyrənmə sürətini hər dövr əvəzinə hər addımda yeniləyir. Məsələn, əvvəllər təyin etdiyimiz `exponential_decay_fn()` funksiyası ilə eyni eksponensial planı necə tətbiq etmək olar:

```
import math

batch_size = 32
n_epochs = 25
n_steps = n_epochs * math.ceil(len(X_train) / batch_size)
scheduled_learning_rate = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.01,
    decay_steps=n_steps,
    decay_rate=0.1)

optimizer = tf.keras.optimizers.SGD(learning_rate=scheduled_learning_rate)
```

Bu, gözəl və sadədir, üstəlik modeli saxladığınız zaman, öyrənmə sürəti və onun planı (vəziyyəti də daxil olmaqla) birlikdə saxlanılır.

1 cycle-ə gəldikdə, Keras onu dəstəkləmir, lakin hər iterasiyada öyrənmə sürətini dəyişdirən xüsusi bir callback yaratmaqla 30 sətirdən az kodla tətbiq etmək mümkündür. Callback-in `on_batch_begin()` metodundan optimizatorun öyrənmə sürətini yeniləmək üçün `tf.keras.backend.set_value(self.model.optimizer.learning_rate,`

`new_learning_rate`) çağırmalısınız. Bir nümunə üçün notebook-un "1Cycle Scheduling" bölməsinə baxın.

Xülasə, exponential decay, performance scheduling və 1cycle yığılmanı əhəmiyyətli dərəcədə sürətləndirə bilər, buna görə onları sınayın!

Regularizasiya ilə Overfitting-dən Qaçınma

"Dörd parametrlə bir filə sığdırı bilərəm və beşlə onun xortumunu silkələyə bilərəm."

— John von Neumann, Enrico Fermi tərəfindən *Nature* 427-də sitat gətirilmişdir

Minlərlə parametrlə bütün zooparkı sığdırı bilərsiniz. Dərin neyron şəbəkələr adətən on minlərlə, bəzən isə milyonlarla parametrlə malikdir. Bu, onlara inanılmaz dərəcədə çox sərbəstlik verir və böyük müxtəliflikdə mürəkkəb dataset-ləri uyğunlaşdırı biləcəkləri deməkdir. Lakin bu böyük çeviklik şəbəkəni təlim setini aşırı uyğunlaşdırmağa (overfitting) meyilli edir. Bunun qarşısını almaq üçün tez-tez regularizasiya lazımdır.

Biz Fəsil 10-da ən yaxşı regularizasiya texnikalarından birini artıq tətbiq etmişik: early stopping. Üstəlik, batch normalization qeyri-sabit qradientlər problemlərini həll etmək üçün nəzərdə tutulsa da, eyni zamanda olduqca yaxşı bir regularizator kimi də çıxış edir. Bu bölmədə neyron şəbəkələr üçün digər populyar regularizasiya texnikalarını nəzərdən keçirəcəyik: ℓ_1 və ℓ_2 regularizasiya, dropout və max-norm regularizasiya.

ℓ_1 və ℓ_2 Regularizasiya

Fəsil 4-də sadə xətti modellər üçün etdiyiniz kimi, neyron şəbəkənin birləşmə çəkirlərini məhdudlaşdırmaq üçün ℓ_2 regularizasiyasından və/və ya seyrək model istəyirsinizsə (çoxlu çəki 0-a bərabər olan) ℓ_1 regularizasiyasından istifadə edə bilərsiniz. Keras qatının birləşmə çəkirlərinə 0.01 regularizasiya amili ilə ℓ_2 regularizasiyasını necə tətbiq etmək olar:

```
layer = tf.keras.layers.Dense(100, activation="relu",  
kernel_initializer="he_normal",
```

```
kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

`l2()` funksiyası təlim zamanı hər addımda regularizasiya itkisini hesablamaq üçün çağırılacaq bir regularizator qaytarır. Bu, son itkiyə əlavə edilir. Təxmin etdiyiniz kimi, ℓ_1 regularizasiyası üçün `tf.keras.regularizers.l1()`, hər iki ℓ_1 və ℓ_2 regularizasiyasını istəyirsinizsə, `tf.keras.regularizers.l1_l2()` (hər iki regularizasiya amilini təyin edərək) istifadə edə bilərsiniz.

Adətən, şəbəkədəki bütün qatlara eyni regularizatoru tətbiq etmək, həmçinin bütün gizli qatlarda eyni aktivasiya funksiyasından və eyni ilkinləşdirmə strategiyasından istifadə etmək istəyəcəksiniz, buna görə eyni arqumentləri təkrarladığınızı görə bilərsiniz. Bu, kodu çirkin və səhvə meyilli edir. Bunun qarşısını almaq üçün kodunuzu loop-lardan istifadə edəcək şəkildə yenidən qurmağa çalışa bilərsiniz. Başqa bir seçim, Python-un `functools.partial()` funksiyasından istifadə etməkdir. Bu, istənilən callable üçün bəzi defolt arqument dəyərləri ilə nazik bir qatı (wrapper) yaratmağa imkan verir:

```
from functools import partial

RegularizedDense = partial(tf.keras.layers.Dense,
                           activation="relu",
                           kernel_initializer="he_normal",

kernel_regularizer=tf.keras.regularizers.l2(0.01))

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    RegularizedDense(100),
    RegularizedDense(100),
    RegularizedDense(10, activation="softmax")
])
```

XƏBƏRDARLIQ

Daha əvvəl gördüyümüz kimi, ℓ_2 regularizasiyası SGD, momentum optimallaşdırması və Nesterov momentum optimallaşdırması ilə yaxşıdır, lakin Adam və onun variantları ilə deyil. Əgər Adam-ı weight decay ilə istifadə etmək istəyirsinizsə, ℓ_2 regularizasiyasından istifadə etməyin: əvəzində AdamW istifadə edin.

Dropout

Dropout dərin neyron şəbəkələr üçün ən populyar regularizasiya texnikalarından biridir. 2012-ci ildə Geoffrey Hinton və digərlərinin məqaləsində təklif edilmişdir və

2014-cü ildə Nitish Srivastava və digərlərinin məqaləsində daha da təfərrüatlandırılmışdır. O, çox uğurlu olduğunu sübut etdi: bir çox müasir neyron şəbəkələri dropout istifadə edir, çünki o, onlara 1%–2% dəqiqlik artımı verir. Bu, çox kimi səslənməyə bilər, lakin model artıq 95% dəqiqliyə malik olduqda, 2% dəqiqlik artımı səhv nisbətini demək olar ki, 40% azaltmaq deməkdir (5% səhvdən təxminən 3% səhvə).

Bu, olduqca sadə bir alqoritmdir: hər bir təlim addımında hər bir neyron (giriş neyronları da daxil olmaqla, lakin həmişə çıxış neyronları istisna olmaqla) p ehtimalı ilə müvəqqəti "düşürülür", yəni bu təlim addımında tamamilə nəzərə alınmır, lakin növbəti addımda aktiv ola bilər (bax Şəkil 11-10). Hiperparametr p *dropout rate* adlanır və adətən 10% ilə 50% arasında təyin edilir: recurrent neural nets-də (bax Fəsil 15) 20%–30%-ə, convolutional neural networks-lərdə isə (bax Fəsil 14) 40%–50%-ə yaxın. Təlimdən sonra neyronlar artıq düşürülmür. Və hamısı budur (bir texniki detal istisna olmaqla, onu qısa müddətdə müzakirə edəcəyik).

İlk baxışda bu dağıdıcı texnikanın işləməsi təəccüblüdür. Bir şirkət, əgər işçilərə hər səhər işə getmək və ya getməmək barədə qərar vermək üçün bir qəpik atmaq tapşırılsa, daha yaxşı işləyərmi? Yaxşı, kim bilir, bəlkə də! Şirkət öz təşkilatını uyğunlaşdırmağa məcbur olardı; o, qəhvə maşınına nəzarət etmək və ya digər kritik tapşırıqları yerinə yetirmək üçün heç bir fərdi şəxsə güvənə bilməzdi, buna görə bu təcrübə bir neçə insan arasında yayılmalı idi. İşçilər yalnız bir neçə həmkarı ilə deyil, bir çox həmkarı ilə əməkdaşlıq etməyi öyrənməli idilər. Şirkət çox daha dayanıqlı olardı. Əgər bir nəfər işdən çıxsa, bu çox da fərq yaratmazdı. Bu fikrin şirkətlər üçün həqiqətən işləyəcəyi aydın deyil, lakin neyron şəbəkələr üçün mütləq işləyir. Dropout ilə öyrədilən neyronlar qonşu neyronları ilə birlikdə uyğunlaşa bilməzlər; öz başlarına mümkün qədər faydalı olmalıdırlar. Onlar həmçinin yalnız bir neçə giriş neyronuna həddən artıq güvənə bilməzlər; hər bir giriş neyronuna diqqət yetirməlidirlər. Nəticədə, girişlərdəki kiçik dəyişikliklərə daha az həssas olurlar. Sonda, daha möhkəm və daha yaxşı ümumiləşən bir şəbəkə əldə edirsiniz.

Şəkil 11-10. Dropout regularizasiyası ilə, hər bir təlim iterasiyasında bir və ya bir neçə qatdakı bütün neyronların təsadüfi alt çoxluğu (çıkış qatı istisna olmaqla) "düşürülür"; bu neyronlar bu iterasiyada 0 çıxarır (kəsikli oxlarla təmsil olunur)

Dropout-un gücünü anlamağın başqa bir yolu, hər bir təlim addımında unikal bir neyron şəbəkəsinin yaradılmasını dərk etməkdir. Hər bir neyron ya mövcud, ya da olmadığı üçün, ümumilikdə 2^N mümkün şəbəkə var (burada N düşürülə bilən neyronların ümumi sayıdır). Bu o qədər böyük bir rəqəmdir ki, eyni neyron şəbəkəsinin iki dəfə nümunə olması demək olar ki, mümkün deyil. 10,000 təlim addımı işlətdikdən sonra, hər biri yalnız bir təlim instansı ilə 10,000 müxtəlif neyron şəbəkəsinə öyrətmişsiniz deməkdir. Bu neyron şəbəkələri açıq-aydın müstəqil deyillər, çünki onlar bir çox çəkirləri paylaşırırlar, lakin buna baxmayaraq, hamısı fərqlidir.

Nəticədə yaranan neyron şəbəkəsi bütün bu kiçik neyron şəbəkələrinin ortalama ansamblı kimi görünə bilər.

İPUCU

Praktikada, adətən dropout yalnız yuxarı bir-üç qatdakı neyronlara tətbiq edə bilərsiniz (çıxış qatı istisna olmaqla).

Bir kiçik, lakin vacib texniki detal var. Tutaq ki, $p = 75\%$: orta hesabla təlim zamanı hər addımda neyronların yalnız 25%-i aktivdir. Bu o deməkdir ki, təlimdən sonra bir neyron təlim zamanı olduğundan dörd dəfə çox giriş neyronuna qoşulmuş olacaq. Bunu kompensasiya etmək üçün, təlim zamanı hər bir neyronun giriş əlaqə çəkilərini dördə vurmaliyiq. Əgər etməsək, neyron şəbəkəsi yaxşı çıxış etməyəcək, çünki təlim zamanı və sonra fərqli məlumatlar görəcək. Daha ümumi olaraq, təlim zamanı birləşmə çəkilərini *keep probability* ($1 - p$)-yə bölməliyik.

Dropout-u Keras-da tətbiq etmək üçün `tf.keras.layers.Dropout` qatından istifadə edə bilərsiniz. Təlim zamanı o, təsadüfi olaraq bəzi girişləri düşürür (onları 0-a təyin edir) və qalan girişləri *keep probability*-yə bölür. Təlimdən sonra o, heç nə etmir; sadəcə girişləri növbəti qata ötürür. Aşağıdakı kod hər bir sıx qatdan əvvəl, 0.2 dropout rate ilə dropout regularizasiyası tətbiq edir:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

```
[...] # compile and train the model
```

XƏBƏRDARLIQ

Dropout yalnız təlim zamanı aktiv olduğundan, təlim itkisi ilə validasiya itkisini müqayisə etmək yanıltıcı ola bilər. Xüsusilə, model təlim setini aşırı uyğunlaşdırma və buna baxmayaraq oxşar təlim və validasiya itkilərinə malik ola bilər. Buna görə, təlim itkisini dropout olmadan qiymətləndirdiyinizə əmin olun (məsələn, təlimdən sonra).

Əgər modelin overfitting etdiyini müşahidə etsəniz, dropout rate-i artırma bilərsiniz. Əksinə, əgər model training set-i underfit edirsə, dropout rate-i azaltmağa çalışmalısınız. Böyük qatlar üçün dropout rate-i artırmaq, kiçik olanlar üçün azaltmaq da kömək edə bilər. Üstəlik, bir çox müasir arxitekturalar dropout-u yalnız son gizli qatdan sonra istifadə edir, buna görə tam dropout çox güclüdirsə, bunu sınaq etmək istəyə bilərsiniz.

Dropout yığılmanı əhəmiyyətli dərəcədə yavaşlatmağa meyllidir, lakin düzgün tənzimləndikdə çox vaxt daha yaxşı bir model ilə nəticələnir. Buna görə, xüsusən böyük modellər üçün, əlavə vaxt və səy adətən buna dəyər.

İPUCU

Əgər SELU aktivasiya funksiyasına əsaslanan self-normalizing bir şəbəkəni regularize etmək istəyirsinizsə (əvvəllər müzakirə edildiyi kimi), *alpha dropout* istifadə etməlisiniz: bu, girişlərin ortalama və standart sapmasını qoruyan dropout variantıdır. O, SELU ilə eyni məqalədə təqdim edilmişdir, çünki adi dropout self-normalizasiyanı poza bilər.

Monte Carlo (MC) Dropout

2016-cı ildə Yarın Gal və Zoubin Ghahramani-nin məqaləsi dropout istifadəsi üçün daha bir neçə yaxşı səbəb əlavə etdi:

- Birincisi, məqalə dropout şəbəkələri (yəni Dropout qatlarını ehtiva edən neyron şəbəkələri) ilə təqribi Bayesian çıxarma (approximate Bayesian inference) arasında dərin bir əlaqə qurdu və dropout-a möhkəm riyazi əsaslandırma verdi.
- İkincisi, müəlliflər *MC dropout* adlı güclü bir texnika təqdim etdilər. Bu, hər hansı öyrədilmiş dropout modelinin performansını onu yenidən öyrətmədən və ya hətta dəyişdirmədən artırma bilər. O, həmçinin modelin qeyri-müəyyənliyinin daha yaxşı ölçüsünü təmin edir və bir neçə sətir kodla tətbiq edilə bilər.

Əgər bunların hamısı "bir qəribə hiylə" kimi səslənsə, aşağıdakı koda baxın. Bu, MC dropout-un tam tətbiqidir, daha əvvəl öyrətdiyimiz dropout modelini yenidən öyrətmədən gücləndirir:

```
import numpy as np
```

```
y_probas = np.stack([model(X_test, training=True) for sample in  
range(100)])
```

```
y_proba = y_probas.mean(axis=0)
```

Qeyd edək ki, `model(X)` `model.predict(X)`-ə bənzəyir, lakin tensor qaytarır, NumPy massivi deyil, və `training` argumentini dəstəkləyir. Bu kod nümunəsində, `training=True` təyin etmək Dropout qatının aktiv qalmasını təmin edir, beləliklə bütün proqnozlar bir qədər fərqli olacaq. Biz test seti üzərində 100 proqnoz edirik və onların ortalamasını hesablayırıq. Daha dəqiq desək, modelə hər çağırış hər instans üçün bir sıra və hər sinif üçün bir sütun olan bir matris qaytarır. Test setində 10,000 instans və 10 sinif olduğundan, bu, [10000, 10] formasında bir matrisdir. Biz 100 belə matrisi yığıraq, beləliklə `y_probas` [100, 10000, 10] formasında 3D massivdir. Birinci ölçü üzərində (`axis=0`) ortalama aldıqda, `y_proba` alırıq, [10000, 10] formasında bir massiv, tək bir proqnozla əldə edəcəyimiz kimi. Hamısı budur! Dropout açıq olduğu halda çoxsaylı proqnozlar üzərində ortalama almaq, bizə adətən dropout söndürülmüş tək bir proqnozun nəticəsindən daha etibarlı olan bir Monte Carlo təxmini verir. Məsələn, Fashion MNIST test setindəki birinci instans üçün modelin proqnozuna baxaq, dropout söndürülmüş halda:

```
>>> model.predict(X_test[:1]).round(3)
```

```
array([[0.    , 0.    , 0.    , 0.    , 0.    , 0.024, 0.    , 0.132, 0.    ,
        0.844]], dtype=float32)
```

Model bu şəklin 9-cu sinfə (topuq botu) aid olduğuna kifayət qədər əmindir (84.4%). Bunu MC dropout proqnozu ilə müqayisə edin:

```
>>> y_proba[0].round(3)
```

```
array([0.    , 0.    , 0.    , 0.    , 0.    , 0.067, 0.    , 0.209, 0.001,
        0.723], dtype=float32)
```

Model hələ də 9-cu sinfə üstünlük verir, lakin onun inamı 72.3%-ə düşdü və 5-ci (sandalet) və 7-ci (krossovka) siniflər üçün təxmin edilən ehtimallar artdı, bu da ayaqqabı olduqları üçün məntiqlidir.

MC dropout modelin ehtimal təxminlərinin etibarlılığını yaxşılaşdırmağa meyllidir. Bu o deməkdir ki, onun əmin olmaqla səhv etmə ehtimalı daha azdır, bu da təhlükəli ola bilər: təsəvvür edin ki, özü idarə edən bir avtomobil dayanma işarəsini etibarlı şəkildə nəzərə almır. Digər siniflərin hansının ən çox ehtimal olunduğunu dəqiq bilmək də faydalıdır. Əlavə olaraq, ehtimal təxminlərinin standart sapmasına baxa bilərsiniz:

```
>>> y_std = y_probas.std(axis=0)
```

```
>>> y_std[0].round(3)
```

```
array([0.    , 0.    , 0.    , 0.001, 0.    , 0.096, 0.    , 0.162, 0.001,
0.183], dtype=float32)
```

Göründüyü kimi, 9-cu sinif üçün ehtimal təxminlərində kifayət qədər çox dispersiya var: standart sapma 0.183-dür, bu, təxmin edilən ehtimal 0.723 ilə müqayisə edilməlidir: əgər riskə həssas bir sistem qururdunuzsa (məsələn, tibbi və ya maliyyə sistemi), yəqin ki, belə qeyri-müəyyən bir proqnoza son dərəcə ehtiyatla yanaşardınız. Siz onu mütləq 84.4% əmin proqnoz kimi qəbul etməzdiniz. Modelin dəqiqliyi həmçinin (çox) kiçik bir artım əldə etdi, 87.0%-dən 87.2%-ə:

```
>>> y_pred = y_proba.argmax(axis=1)
>>> accuracy = (y_pred == y_test).sum() / len(y_test)
>>> accuracy
```

```
0.8717
```

QEYD

İstifadə etdiyiniz Monte Carlo nümunələrinin sayı (bu nümunədə 100) tənzimləyə biləcəyiniz bir hiperparametrdir. Nə qədər çox olarsa, proqnozlar və onların qeyri-müəyyənlik təxminləri bir o qədər dəqiq olacaq. Lakin onu ikiqat artırırsınız, çıxarma vaxtı da ikiqat artacaq. Üstəlik, müəyyən sayda nümunədən yuxarı, az təkmilləşmə görəcəksiniz. Sizin işiniz tətbiqinizdən asılı olaraq gecikmə və dəqiqlik arasında düzgün mübadiləni tapmaqdır.

Əgər modelinizdə təlim zamanı xüsusi davranan digər qatlar varsa (məsələn, BatchNormalization qatları), onda bizim etdiyimiz kimi təlim rejimini məcbur etməməlisiniz. Əvəzində, Dropout qatlarını aşağıdakı `MCDropout` sinfi ilə əvəz etməlisiniz:

```
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=False):

        return super().call(inputs, training=True)
```

Burada biz sadəcə Dropout qatını alt sinifə çeviririk və `call()` metodunu onun `training` argumentini `True`-a məcbur etmək üçün override edirik (bax Fəsil 12). Eynilə, `AlphaDropout` əvəzinə `MCAAlphaDropout` sinfini təyin edə bilərsiniz. Əgər sıfırdan bir model yaradırsınızsa, sadəcə Dropout əvəzinə `MCDropout` istifadə etmək lazımdır. Lakin əgər Dropout istifadə edərək artıq öyrədilmiş bir modeliniz varsa, mövcud modelə bənzər, lakin Dropout əvəzinə `MCDropout` olan yeni bir model yaratmalı və mövcud modelin çəkirlərini yeni modelə köçürməlisiniz.

Qısacası, MC dropout, dropout modellərini gücləndirən və daha yaxşı qeyri-müəyyənlik təxminləri təmin edən əla bir texnikadır. Və əlbəttə, təlim zamanı adi dropout olduğundan, o, həmçinin regularizator kimi də çıxış edir.

Max-Norm Regularizasiya

Neyron şəbəkələr üçün digər bir məşhur regularizasiya texnikası *max-norm regularization* adlanır: hər bir neyron üçün, gələn əlaqələrin çəkirlərini w elə məhdudlaşdırır ki, $\|w\|_2 \leq r$ olsun, burada r max-norm hiperparametri və $\|\cdot\|_2$ ℓ_2 normasıdır.

Max-norm regularizasiyası ümumi itki funksiyasına regularizasiya itki termini əlavə etmir. Bunun əvəzinə, adətən hər bir təlim addımından sonra $\|w\|_2$ hesablanmaqla və lazım olduqda w yenidən miqyaslandırmaqla həyata keçirilir ($w \leftarrow w / \|w\|_2$).

r -ni azaltmaq regularizasiyanın miqdarını artırır və overfitting-i azaltmağa kömək edir. Max-norm regularizasiyası həmçinin qeyri-sabit qradientlər problemlərini yüngülləşdirməyə kömək edə bilər (əgər batch normalization istifadə etmirsinizsə).

Max-norm regularizasiyasını Keras-da tətbiq etmək üçün, hər bir gizli qatın `kernel_constraint` argumentini müvafiq max dəyəri ilə `max_norm()` constraint-ə təyin edin:

```
dense = tf.keras.layers.Dense(100, activation="relu",  
                                kernel_initializer="he_normal",
```

```
kernel_constraint=tf.keras.constraints.max_norm(1.))
```

Hər bir təlim iterasiyasından sonra, modelin `fit()` metodu `max_norm()` tərəfindən qaytarılan obyekti çağıraraq, ona qatın çəkirlərini ötürəcək və əvəzində yenidən miqyaslandırılmış çəkilər alacaq, sonra bunlar qatın çəkirlərini əvəz edəcək. Fəsil 12-də görəcəyiniz kimi, lazım olduqda öz xüsusi constraint funksiyanızı təyin edə və onu `kernel_constraint` kimi istifadə edə bilərsiniz. Siz həmçinin `bias_constraint` argumentini təyin etməklə bias terminlərini də məhdudlaşdırı bilərsiniz.

`max_norm()` funksiyasının defolt olaraq 0 olan bir `axis` argumenti var. Dense qatı adətən [number of inputs, number of neurons] formasında çəkilərə malikdir, buna görə `axis=0` istifadə etmək max-norm constraint-inin hər bir neyronun çəki vektoruna müstəqil olaraq tətbiq ediləcəyi deməkdir. Əgər max-norm-u convolutional layers (bax Fəsil 14) ilə istifadə etmək istəyirsinizsə, `max_norm()` constraint-inin `axis` argumentini müvafiq olaraq təyin etdiyinizə əmin olun (adətən `axis=[0, 1, 2]`).

Xülasə və Praktik Göstərişlər

Bu fəsildə geniş bir texnika spektrini əhatə etdik və onlardan hansını istifadə edəcəyinizi düşünə bilərsiniz. Bu, tapşırıqdan asılıdır və hələ də aydın bir konsensus yoxdur, lakin mən Cədvəl 11-3-dəki konfigurasiyanın əksər hallarda çox hiperparametr tənzimlənməsi tələb etmədən yaxşı işlədiyini tapmışam. Bununla belə, bu defoltları sərt qaydalar kimi qəbul etməyin!

Cədvəl 11-3. Defolt DNN konfigurasiyası

Hiperparametr	Defolt dəyər
Kernel ilkinləşdiricisi	He initialization
Aktivasiya funksiyası	Dayazdırsa ReLU; dərinirsə Swish
Normallaşdırma	Dayazdırsa yox; dərinirsə batch norm
Regularizasiya	Early stopping; lazım olduqda weight decay
Optimizator	Nesterov accelerated gradients və ya AdamW
Öyrənmə sürəti planı	Performance scheduling və ya 1 cycle

Şəbəkə sadə sıx qatlar yığınıdırsa, o, self-normalize ola bilər və bunun əvəzinə Cədvəl 11-4-dəki konfigurasiyanı istifadə etməlisiniz.

Cədvəl 11-4. Self-normalizing şəbəkə üçün DNN konfigurasiyası

Hiperparametr	Defolt dəyər
Kernel ilkinləşdiricisi	LeCun initialization
Aktivasiya funksiyası	SELU
Normallaşdırma	None (self-normalization)

Regularizasiya	Lazım olduqda Alpha dropout
Optimizator	Nesterov accelerated gradients
Öyrənmə sürəti planı	Performance scheduling və ya 1cycle

Giriş xüsusiyyətlərini normallaşdırmağı unutmayın! Həmçinin, əgər oxşar problemi həll edən əvvəlcədən öyrədilmiş bir neyron şəbəkəsi tapa bilsəniz, onun hissələrini təkrar istifadə etməyə çalışın, və ya çoxlu etiketsiz məlumatınız varsa unsupervised pretraining, və ya oxşar tapşırıq üçün çoxlu etiketli məlumatınız varsa köməkçi tapşırıqda pretraining istifadə edin.

Əvvəlki göstərişlər əksər halları əhatə etməlidir, lakin bəzi istisnalar:

- Seyrək bir modelə ehtiyacınız varsa, ℓ_1 regularizasiyasından (və istəyə bağlı olaraq təlimdən sonra kiçik çəkiləri sıfırlamaqdan) istifadə edə bilərsiniz. Daha da seyrək bir modelə ehtiyacınız varsa, TensorFlow Model Optimization Toolkit-dən istifadə edin. Bu, self-normalizasiyanı pozacaq, buna görə bu halda defolt konfigurasiyadan istifadə etməlisiniz.
- Aşağı gecikməli (ildırım sürəti ilə proqnozlar verən) bir modelə ehtiyacınız varsa, daha az qat, ReLU və ya leaky ReLU kimi sürətli aktivasiya funksiyası istifadə etməli və təlimdən sonra batch normalization qatlarını əvvəlki qatlara birləşdirməlisiniz. Seyrək bir model də kömək edəcək. Nəhayət, float dəqiqliyini 32 bitdən 16 və ya hətta 8 bitə endirmək istəyə bilərsiniz (bax "Modeli Mobil və ya Quraşdırılmış Cihaza Yerləşdirmə"). Yenə TF-MOT-a baxın.
- Riskə həssas bir tətbiq qurursunuzsa və ya çıxarma gecikməsi tətbiqinizdə çox vacib deyilsə, performans artırmaq və daha etibarlı ehtimal təxminləri, həmçinin qeyri-müəyyənlik təxminləri əldə etmək üçün MC dropout istifadə edə bilərsiniz.

Bu göstərişlərlə, artıq çox dərin şəbəkələri öyrətməyə hazırsınız! Ümid edirəm ki, indi sadəcə rahat Keras API-dən istifadə edərək çox şey edə biləcəyinizə inanırsınız. Bununla belə, bir zaman gələcək ki, daha çox nəzarətə ehtiyacınız olacaq; məsələn, xüsusi bir loss funksiyası yazmaq və ya təlim alqoritmini dəyişdirmək üçün. Belə hallar üçün TensorFlow-un aşağı səviyyəli API-sindən istifadə etməli olacaqsınız, bunu növbəti fəsildə görəcəksiniz.

Məşqlər

1. Glorot initialization və He initialization hansı problemi həll etməyi hədəfləyir?

2. He initialization istifadə edərək təsadüfi seçilən eyni dəyərə bütün çəkilişləri ilkinləşdirmək düzgündürmü?
3. Bias terminlərini 0-a ilkinləşdirmək düzgündürmü?
4. Bu fəsilə müzakirə etdiyimiz hər bir aktivasiya funksiyasından hansı hallarda istifadə etmək istərdiniz?
5. SGD optimizatorundan istifadə edərkən momentum hiperparametrini 1-ə çox yaxın (məsələn, 0.99999) təyin etsəniz nə baş verə bilər?
6. Seyrək bir model istehsal etmək üçün üç üsulun adını çəkin.
7. Dropout təlimi yavaşlatırmı? O, çıxarmanı (yəni, yeni instanslar üzərində proqnozlar verməyi) yavaşlatırmı? Bəs MC dropout?
8. CIFAR10 şəkil dataset-i üzərində dərin bir neyron şəbəkəsi öyrətməyə çalışın:
 - a. Hər biri 100 neyrondan ibarət 20 gizli qatlı bir DNN qurun (bu çoxdur, lakin məşqin məqsədi budur). He initialization və Swish aktivasiya funksiyasından istifadə edin.
 - b. Nadam optimallaşdırması və early stopping istifadə edərək, CIFAR10 dataset-i üzərində şəbəkəni öyrədin. Onu `tf.keras.datasets.cifar10.load_data()` ilə yükləyə bilərsiniz. Dataset 60,000 32×32 piksellə rəngli şəkildən (50,000 təlim üçün, 10,000 test üçün) 10 sinifdən ibarətdir, buna görə 10 neyronlu softmax çıxış qatına ehtiyacınız olacaq. Modelin arxitekturasını və ya hiperparametrlərini hər dəfə dəyişdirdikdə düzgün öyrənmə sürətini axtarmağı unutmayın.
 - c. İndi batch normalization əlavə etməyə çalışın və öyrənmə ayrılıqlarını müqayisə edin: əvvəlkindən daha sürətli yığılırmı? Daha yaxşı bir model istehsal edirmi? Təlim sürətinə necə təsir edir?
 - d. Batch normalization-u SELU ilə əvəz etməyə çalışın və şəbəkənin self-normalize olmasını təmin etmək üçün lazımi düzəlişləri edin (yəni, giriş xüsusiyyətlərini standartlaşdırın, LeCun normal ilkinləşdirməsindən istifadə edin, DNN-nin yalnız sıx qatlar ardıcılığından ibarət olduğundan əmin olun və s.).
 - e. Modeli alpha dropout ilə regularize etməyə çalışın. Sonra, modelinizi yenidən öyrətmədən, MC dropout istifadə edərək daha yaxşı dəqiqlik əldə edə biləcəyinizə baxın.
 - f. 1cycle scheduling istifadə edərək modelinizi yenidən öyrədin və onun təlim sürətini və model dəqiqliyini yaxşılaşdırıb-yaxşılaşdırmadığına baxın.

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.